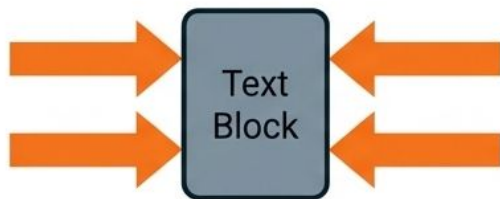


# **Decoder Architecture and Inference in Transformers**

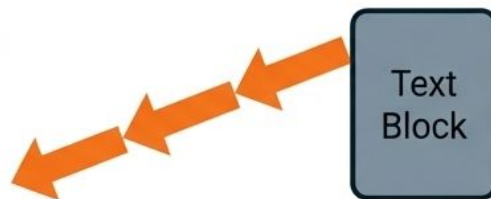
# The Transformer Taxonomy: Three Distinct Architectures

## The Encoder (e.g., BERT)



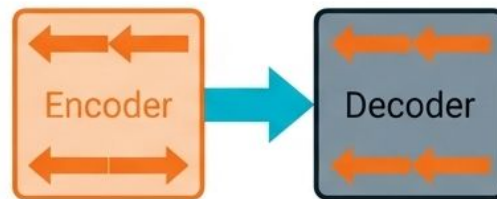
- **Context Flow:** Bidirectional
- **Function:** Transforms text into a rich representation supporting various downstream tasks.
- **Best For:** Understanding, text classification, extraction.

## The Decoder (e.g., GPT-3)



- **Context Flow:** Unidirectional / Autoregressive
- **Function:** Predicts the conditional distribution of the next token.
- **Best For:** Text generation, sequence continuation.

## The Encoder-Decoder



- **Context Flow:** Sequence-to-Sequence
- **Function:** One text string is entirely converted into another.
- **Best For:** Machine translation, summarization.

## **Recap: Masked Multi-Head Attention**

# Enter the Look-Ahead Mask (The $-\infty$ Intervention)

We introduce a structural matrix engineered specifically to act as a mathematical blindfold over the attention scores.

|             | Key (How) | Key (are) | Key (you) |
|-------------|-----------|-----------|-----------|
| Query (How) | Valid     | Leakage   | Leakage   |
| Query (are) | Valid     | Valid     | Leakage   |
| Query (you) | Valid     | Valid     | Valid     |

**Pre-Mask Attention Scores**



|   |           |           |
|---|-----------|-----------|
| 0 | $-\infty$ | $-\infty$ |
| 0 | 0         | $-\infty$ |
| 0 | 0         | 0         |

**The Mask Matrix**

- **Lower Triangle & Diagonal (Past/Present):** Filled with 0. Will not alter the original score upon addition.
- **Upper Triangle (Future/Forbidden):** Filled with  $-\infty$  (Negative Infinity). Designed to mathematically obliterate future connections.

# Applying the Mathematical Blindfold

$$\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}} + \text{MASK}$$



Resulting Masked Matrix

|     |           |           |
|-----|-----------|-----------|
| 112 | $-\infty$ | $-\infty$ |
| 95  | 104       | $-\infty$ |
| 88  | 92        | 115       |

The rule of matrix addition dictates  $\text{Score}_{ij} + (-\infty) = -\infty$ . The model is now mathematically barred from accessing future token states, yet the entire operation was performed strictly in parallel using a single batch calculation.

# The Solution: Masked Self-Attention

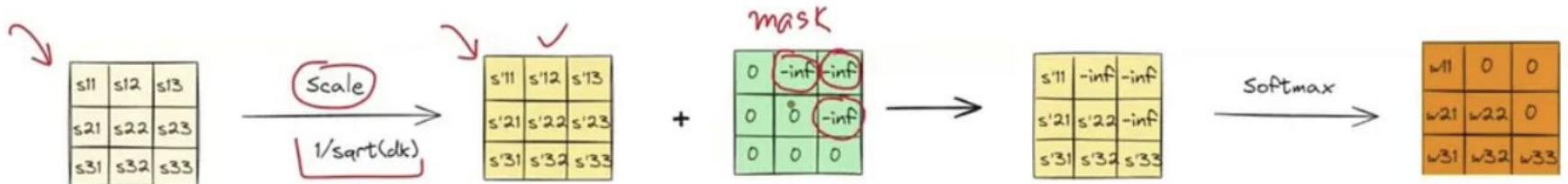
**The Best of Both Worlds:** Masking allows us to keep the speed of parallel, non-autoregressive training while enforcing the strict rules of autoregressive prediction.

**How it Works:** We apply a "Mask Matrix" over the attention scores before they are finalized.

**Negative Infinity:** Values corresponding to future tokens are artificially set to negative infinity ( $-\infty$ ).

$$\begin{aligned}
 \text{आप}_{ce} &= w_{11} * \text{आप} + w_{12} * \text{कैसे} + w_{13} * \text{हैं} \\
 \text{कैसे}_{ce} &= w_{21} * \text{आप} + w_{22} * \text{कैसे} + w_{23} * \text{हैं} \\
 \text{हैं}_{ce} &= w_{31} * \text{आप} + w_{32} * \text{कैसे} + w_{33} * \text{हैं}
 \end{aligned}$$

Handwritten annotations: Red circles around the terms  $w_{12} * \text{कैसे}$  and  $w_{23} * \text{हैं}$  in the equations above, with arrows pointing to them from above.

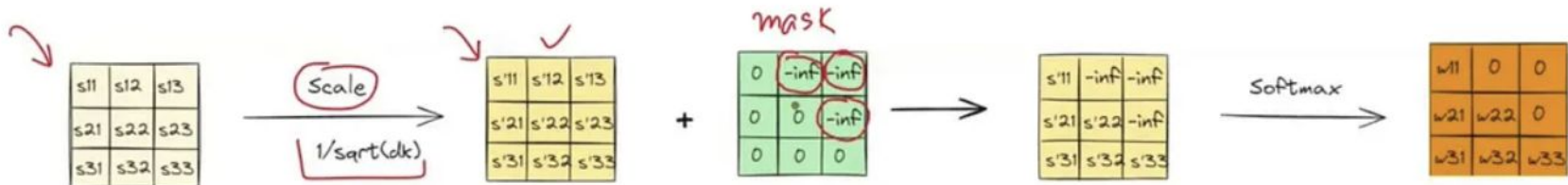


# Softmax and Final Context

**Applying Softmax:** After adding the negative infinity mask, the matrix is passed through the Softmax function.

**Zeroing out the Future:** The Softmax function converts those negative infinity values into exactly zero.

**The Result:** The attention mechanism physically cannot draw any context or "weight" from future tokens, effectively simulating a step-by-step sequential process while calculating everything in parallel.





# The Softmax Neutralization

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

|     |           |           |
|-----|-----------|-----------|
| 112 | $-\infty$ | $-\infty$ |
| 95  | 104       | $-\infty$ |
| 88  | 92        | 115       |

**Masked Score Matrix**



How  
are  
you

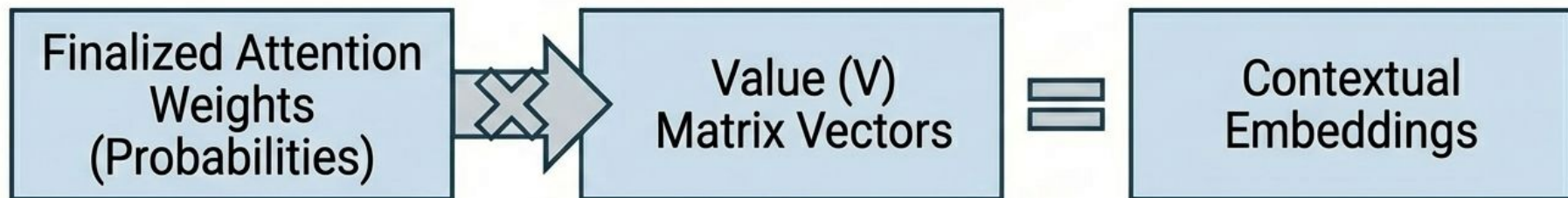
|     |            |            |
|-----|------------|------------|
| 1.0 | <b>0.0</b> | <b>0.0</b> |
| 0.4 | 0.6        | <b>0.0</b> |
| 0.2 | 0.3        | 0.5        |

**Final Attention  
Weights Matrix**

The **Mathematical Magic**: Because  $e^{(-\infty)} = 0$ , the Softmax operation perfectly neutralizes the forbidden future connections, zeroing out their probability weights entirely.



# Generating Safe Contextual Embeddings



## Mathematical Proof of Isolation

$$\text{Context(How)} = 1.0 \times V_{\text{How}} + \mathbf{0.0 \times V_{\text{are}}} + \mathbf{0.0 \times V_{\text{you}}}$$

$$\text{Context(\textbf{are})} = 0.4 \times V_{\text{How}} + 0.6 \times V_{\text{are}} + \mathbf{0.0 \times V_{\text{you}}}$$

The finalized Attention Weights are multiplied by the Value (V) vectors. Because future weights are strictly 0.0, future values are completely excluded from the mathematical sum. Each token's final contextual embedding is **structurally identical** to what it would have been if processed in a slow, sequential Auto-Regressive loop.

# The Masked Attention Masterpiece: Best of Both Worlds

## Masked Multi-Head Attention

### Hardware Optimization (Speed)

All target sequence tokens are processed as a single massive matrix batch.

Training time drops from  $O(N)$  to  $O(1)$  relative to sequence length.

### Mathematical Integrity (No Leakage)

The Look-Ahead Mask ( $-\infty$ ) and Softmax mathematically sever access to future tokens.

The network trains parallelly without cheating.

### Inference Symmetry

Because the training state perfectly mimics the autoregressive inference state mathematically, the model transitions seamlessly from training to real-world prediction without behavior drift.

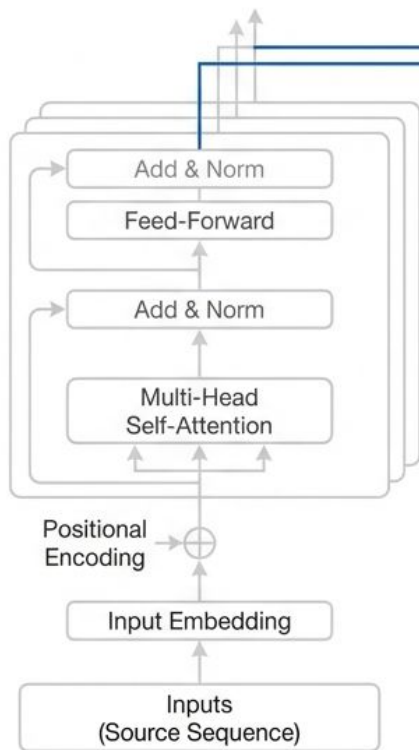
This single structural modification is what enables the massive scale, rapid training efficiency, and sequential accuracy of modern Generative AI.

# Cross-Attention

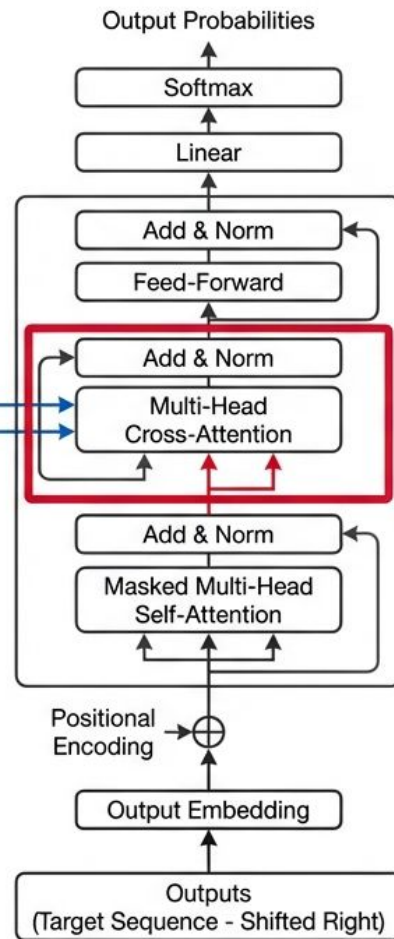
# Locating Cross-Attention in the Transformer Architecture

## Current Lecture Position:

Encoder architecture (Embeddings, Positional Encoding, Self-Attention) and Masked Self-Attention are complete.



**The Architectural Anomaly:** In standard Multi-Head Attention, all three inputs originate from the same sequence. Here, the routing diverges.



**The Intersection:** Two inputs originate from the Encoder's output. One originates from the Decoder. This is the precise junction where source and target sequences intersect.

# Introduction to Cross Attention

**What is it?** Cross attention is a fundamental mechanism in transformers, especially for sequence-to-sequence tasks like machine translation or summarization.

**The Goal:** While self-attention helps a sequence understand itself, cross attention identifies connections *between two different sequences* (e.g., aligning words from English to Hindi).

**The Context:** In an encoder-decoder setup, the encoder reads the input (context), and the decoder generates the output step-by-step. Cross attention allows the decoder to look back at the encoder's input to make accurate predictions.

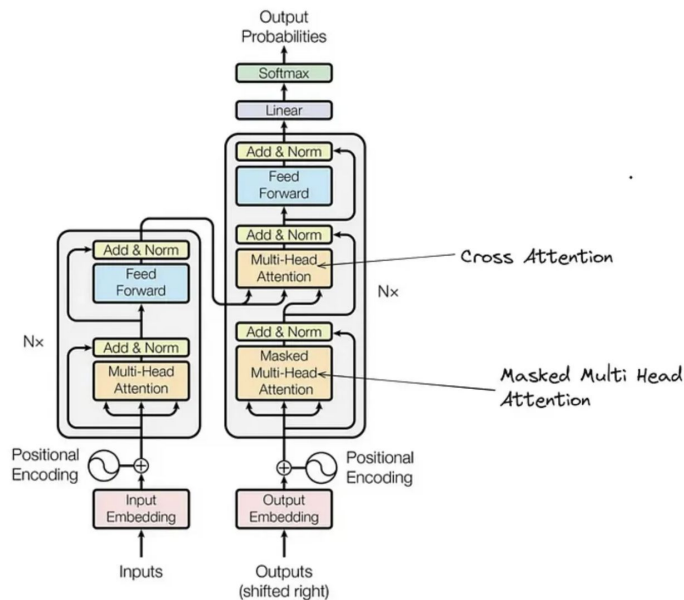


Figure 1: The Transformer - model architecture.

[image from Attention All You Need paper](#)

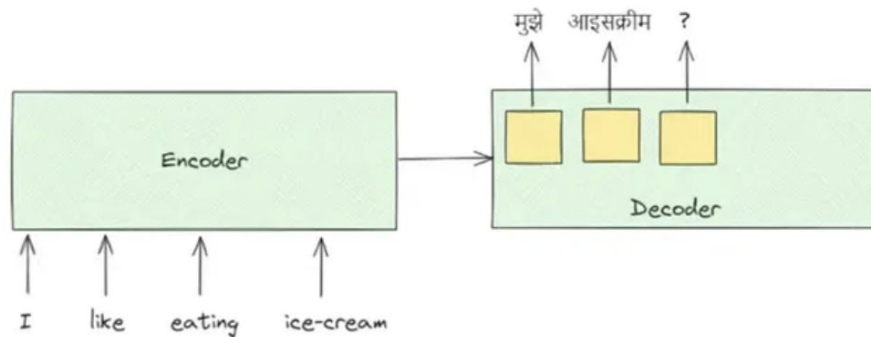


# The Two Factors of Decoding

When generating the next word in a sequence (e.g., translating "I like eating ice cream" to Hindi step-by-step), the decoder relies on two key factors:

1. **What has been generated so far** (e.g., "मुझे" and "आइसक्रीम"). This is handled by *Self-Attention*.
2. **The original input sentence** (e.g., knowing that "eating" corresponds to the next word). This is handled by *Cross-Attention*.

Cross attention allows the decoder to attend to different parts of the input sequence to determine how words in the input relate to words in the output sequence.

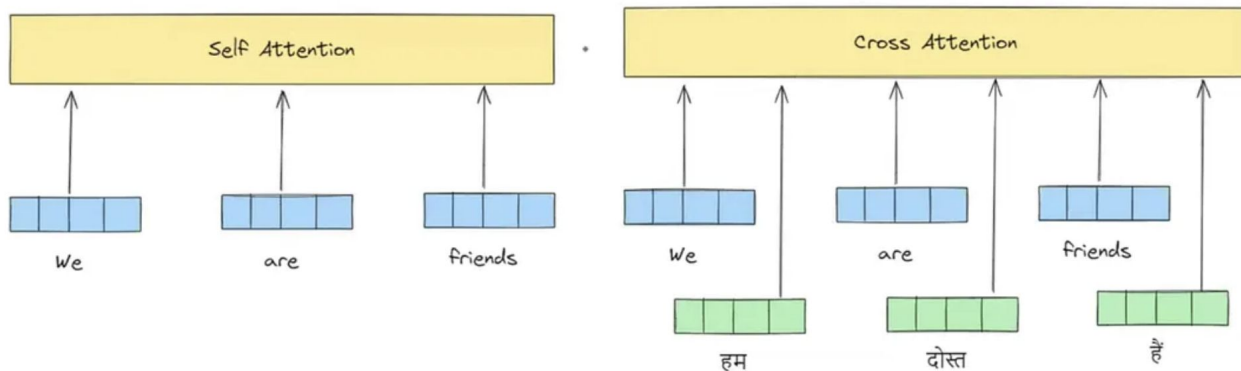


# Self-Attention vs. Cross-Attention (The Input)

**The Core Difference:** While self-attention works on a single sequence, cross attention processes *two* sequences simultaneously.

**Self-Attention Input:** Takes one sequence (e.g., "We are friends") and generates contextual embeddings by finding relationships within that exact same sequence.

**Cross-Attention Input:** Takes both the source sequence ("We are friends") AND the target sequence ("हम दोस्त हैं"). It computes how strongly each word in the source relates to each word in the target.





# Cross Attention: The Processing Mechanism (Q, K, V Vectors)

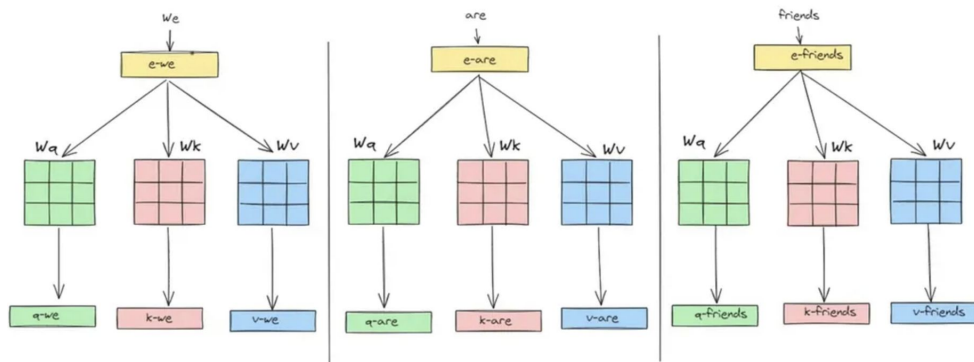
Cross attention is conceptually similar to self-attention but derives its Query (Q), Key (K), and Value (V) vectors differently.

**In Self-Attention:** Q, K, and V all come from the *same* sequence.

**In Cross-Attention:**

- **Query Vectors (Q):** Come from the *output sequence* (e.g., the Hindi target words).
- **Key (K) & Value (V) Vectors:** Come from the *input sequence* (e.g., the English source words).

**The Calculation:** Similarity scores are calculated between the Output Queries and the Input Keys. These scores then weight the Input Values.



# Matrix Projections and Vector Generation

$$\mathbf{E}_{\text{target}} \times \mathbf{W}_Q \rightarrow \mathbf{Q}_{\text{matrix}}$$

$$\mathbf{E}_{\text{source}} \times \mathbf{W}_K \rightarrow \mathbf{K}_{\text{matrix}}$$

$$\mathbf{E}_{\text{source}} \times \mathbf{W}_V \rightarrow \mathbf{V}_{\text{matrix}}$$

**Dimensionality Setup:** Let  $\mathbf{E}_{\text{target}}$  be the embedding from the Decoder's previous layer, and  $\mathbf{E}_{\text{source}}$  be the final output embedding from the Encoder.

**Target Projection:** For every token in the target sequence (e.g., 'Ĥ'), we generate exactly one Query vector via  $\mathbf{W}_Q$ .

**Source Projection:** For every token in the source sequence (e.g., 'We', 'are', 'friends'), we generate one Key vector and one Value vector via  $\mathbf{W}_K$  and  $\mathbf{W}_V$ .

**Crucial Property:** The number of Q vectors does not need to equal the number of K and V vectors. Source and target sequences can be of entirely different lengths.

# Visualizing the Attention Matrix Heatmap



- **Matrix Scaling & Softmax:** Raw dot-product scores are divided by  $\sqrt{d_k}$  (for gradient stability) and normalized via Softmax.
- **Probability Distribution:** This yields a weighted probability distribution across the source sequence for every target token.
- **Interpretation:** The matrix perfectly maps cross-lingual dependencies. The Hindi word 'पसंद' (like) demonstrates massive attention weight on the English word 'like', mathematically bypassing the rest of the sentence context.

# The Output Contextual Embeddings

**Output Size:** In cross-attention, the number of output contextual embeddings corresponds directly to the number of words in the *output sequence*.

**Weighted Contributions:** If the output sequence has three words ("हम," "दोस्त," "हैं"), you receive exactly three contextual embeddings.

**The Formula:** The contextual embedding for "हम" is calculated as a weighted sum of the input embeddings ("We," "are," "friends"), where the weights are determined by the cross-attention similarity scores.

$$ce\_we = 0.8 \times e\_we + 0.1 \times e\_are + 0.1 \times e\_friends$$

$$ce\_are = 0.15 \times e\_we + 0.75 \times e\_are + 0.1 \times e\_friends$$

$$ce\_friends = 0.2 \times e\_we + 0.1 \times e\_are + 0.7 \times e\_friends$$

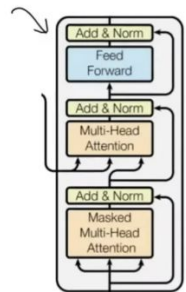
# **Decoder Architecture**

# Introduction to the Transformer Decoder (Training Phase)

**Core Function:** The decoder generates meaningful output sequences (e.g., in machine translation) by leveraging encoder representations and previously generated outputs.

**Training vs. Inference:** During training, the decoder operates in a *non-autoregressive* manner (processing the whole sequence at once), whereas in inference, it becomes *autoregressive*.

**Architecture Overview:** Much like the encoder, the decoder consists of 6 stacked blocks. Each block contains three key components: Masked Self-Attention, Cross-Attention, and a Feed-Forward Neural Network.

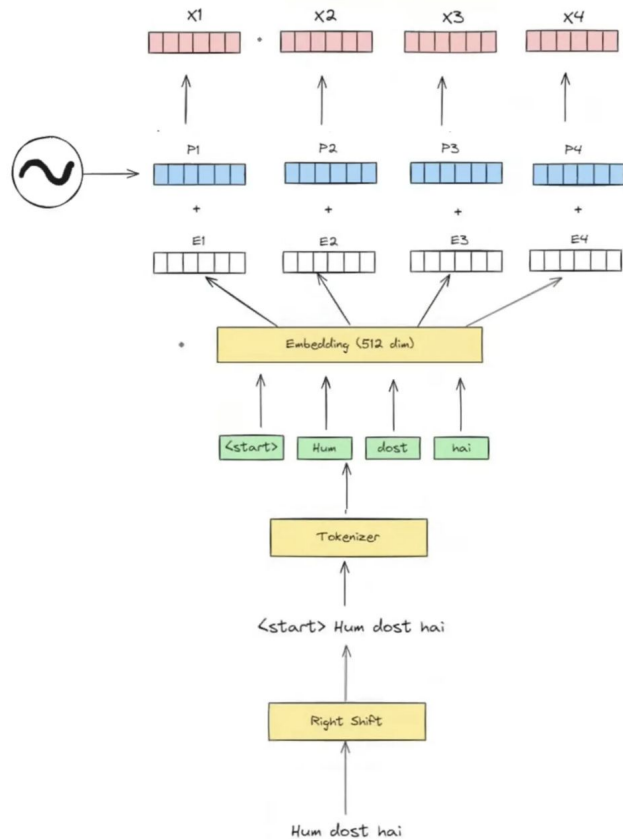




# Input Preparation Phase

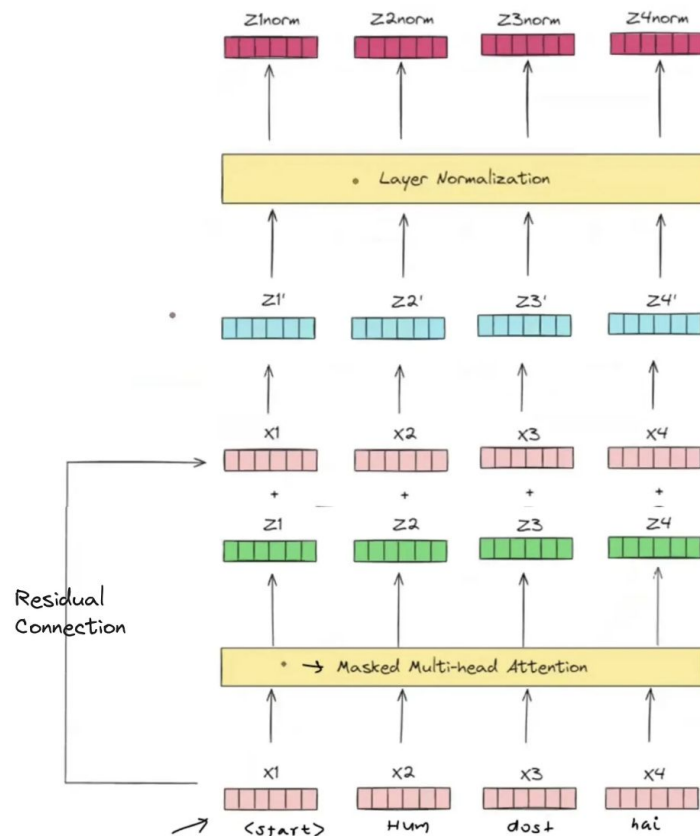
Before entering the decoder block, the target sequence (e.g., the Hindi translation) undergoes four critical transformations:

1. **Right Shifting:** A **<START>** token is added at the beginning to act as a flag indicating the start of the training process.
2. **Tokenization:** Breaking the sequence down into individual tokens (e.g., word-level).
3. **Embedding:** Converting tokens into 512-dimensional numerical vectors.
4. **Positional Encoding:** Adding a unique positional vector to each embedding vector so the model understands the sequential order of the words.



# Masked Multi-Head Attention

1. **The Mechanism:** Operates almost identically to regular multi-head attention, but with a critical difference: *Masking*.
2. **The Intuition:** When generating the contextual embedding for a specific token, the model is only allowed to look at previous tokens and the current token. It strictly ignores (masks) future tokens.
3. **Example:** When processing the second token in a sequence, the attention mechanism considers token 1 and token 2, but masks token 3 and token 4. This prevents the model from "cheating" by looking ahead during training.

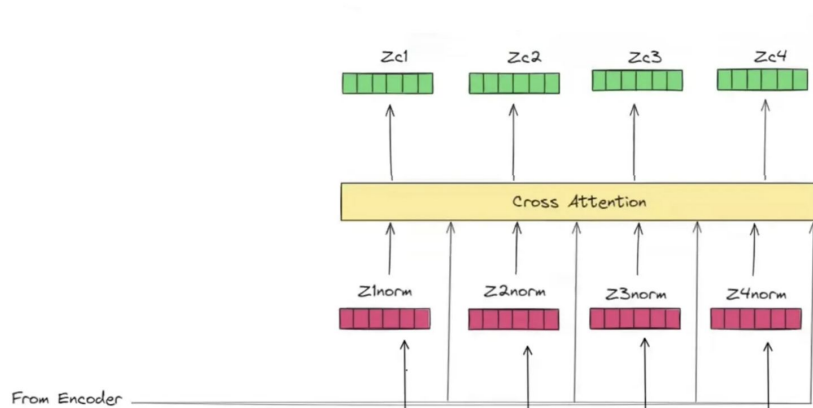


# Cross-Attention (Encoder-Decoder Interaction)

**The Bridge:** This layer allows interaction between the input sequence (e.g., English from the encoder) and the output sequence (e.g., Hindi in the decoder).

**Two Inputs:** 1. The sequence from the preceding Masked Attention block (acts as the Query). 2. The final contextual embeddings from the Encoder (acts as the Key and Value).

**Output:** A new contextual embedding vector for each token that now holds information from both the source language and the target language.



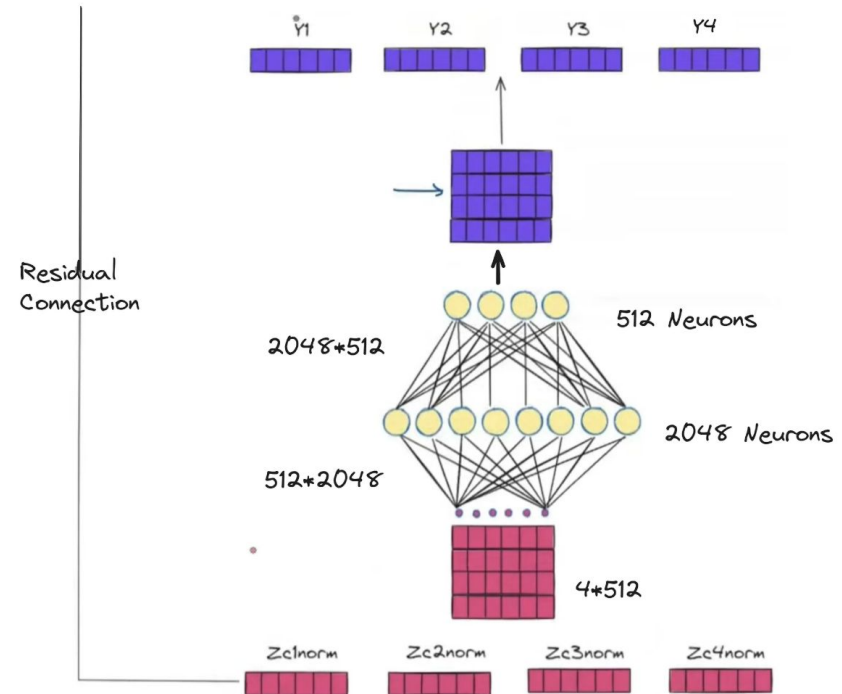
# Feed-Forward Network & Residual Connections

**Add & Normalize (Layer Norm):** After every major attention step, a residual connection bypasses the block, adds the original input to the output, and applies layer normalization. This stabilizes the training process and prevents exploding values.

**Feed-Forward Architecture:** The normalized vectors pass through a two-layer neural network:

- Layer 1: 2048 neurons with a ReLU activation function (introduces non-linearity).
- Layer 2: 512 neurons with a linear activation function.

**Batch Processing:** The 512-dimensional vectors are processed in parallel batches, maintaining their size while learning complex, non-linear representations.



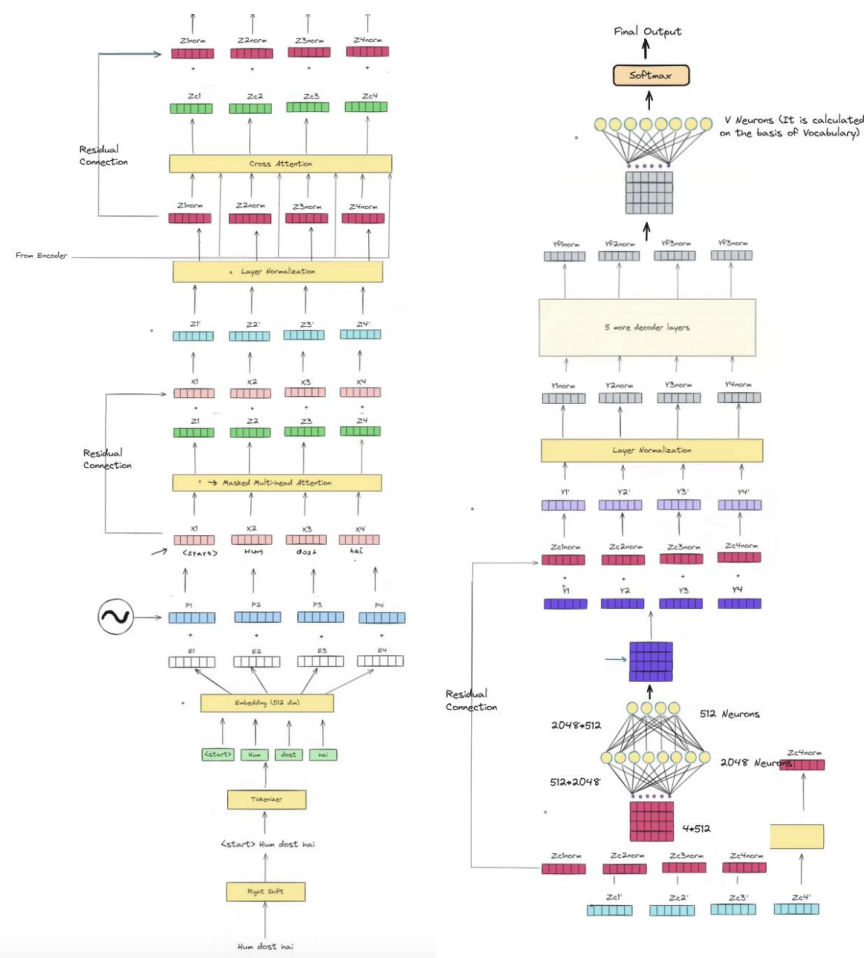
# Output Generation

**Linear Layer:** The final 512-dimensional vectors from the 6th decoder block are passed into a linear layer. The number of neurons here equals the vocabulary size ( $V$ ) of the target dataset.

**Logits to Probabilities:** The linear layer outputs raw scores (logits) for every possible word in the vocabulary.

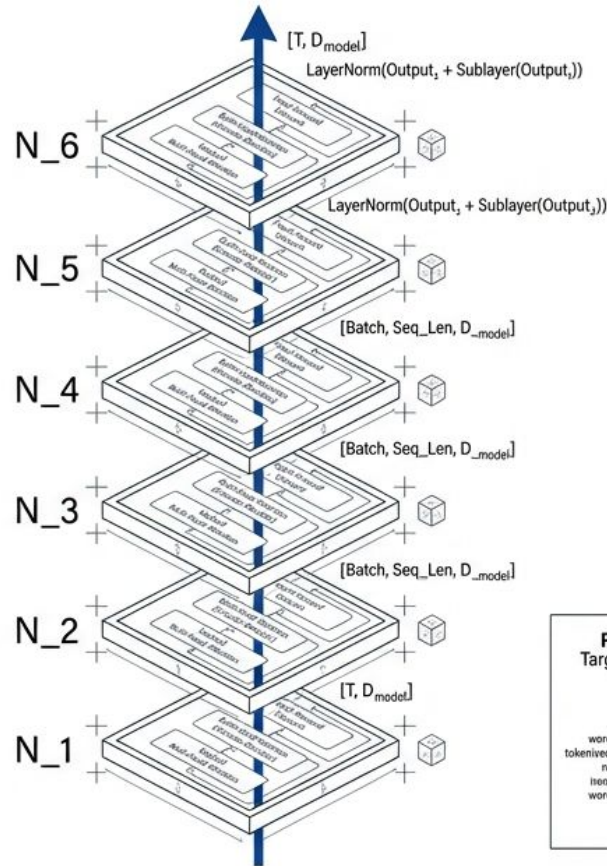
**Softmax Activation:** A Softmax function is applied to normalize these raw scores into a probability distribution (summing to 1).

**Prediction:** The neuron with the highest probability dictates the predicted output word for that specific sequence step.



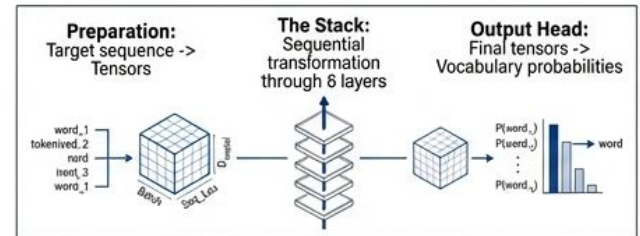
# The N=6 Decoder Stack

Like its Encoder counterpart, the Transformer Decoder is a homogenous stack of identical structural blocks, not a single layer.



## Architectural Characteristics

|                                |                                                                                                                    |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Depth:</b>                  | The architecture utilizes 6 distinct decoder blocks (N=6).                                                         |
| <b>Homogeneity:</b>            | Every block shares the exact same internal wiring and architectural layout.                                        |
| <b>Parameter Independence:</b> | While the structural blueprint is identical, the weights and biases are uniquely learned for each of the 6 blocks. |
| <b>Information Flow:</b>       | The output of Block <i>i</i> acts as the direct input to Block <i>i</i> +1.                                        |

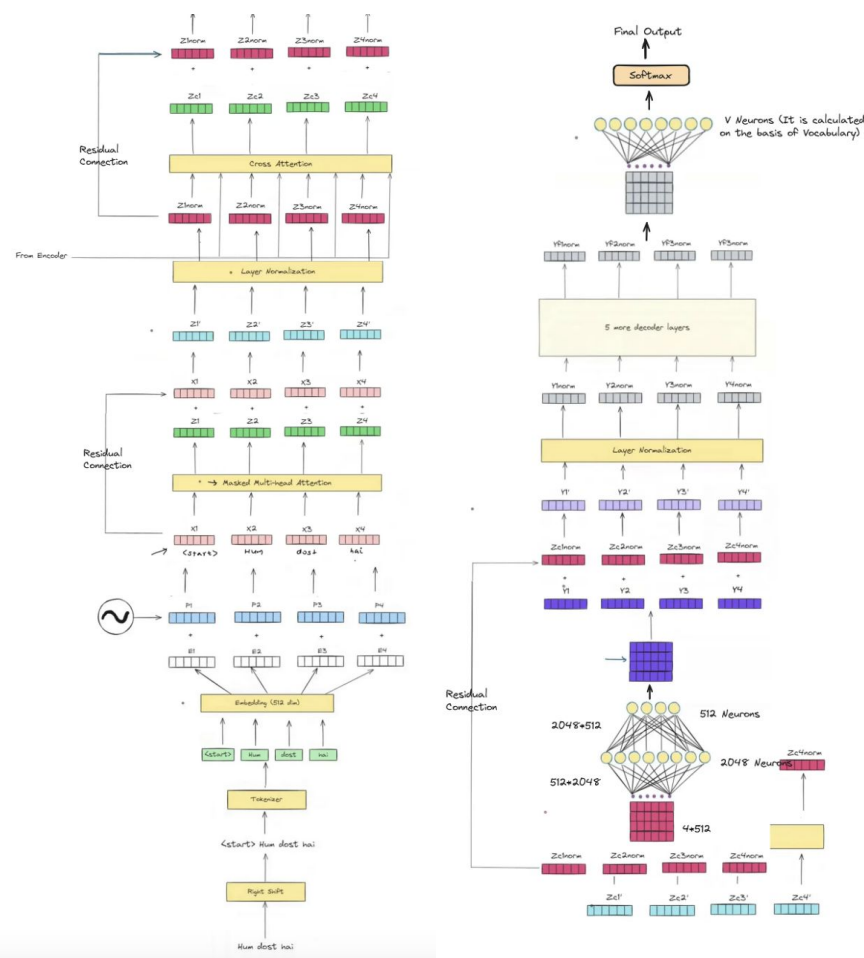




# Decoder Architecture

Just like the Encoder, it is a **stack of multiple layers**. Each layer follows this specific flow:

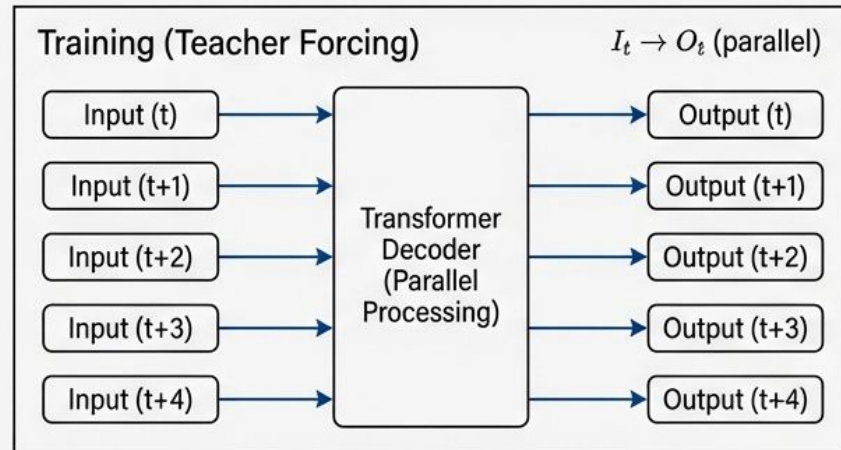
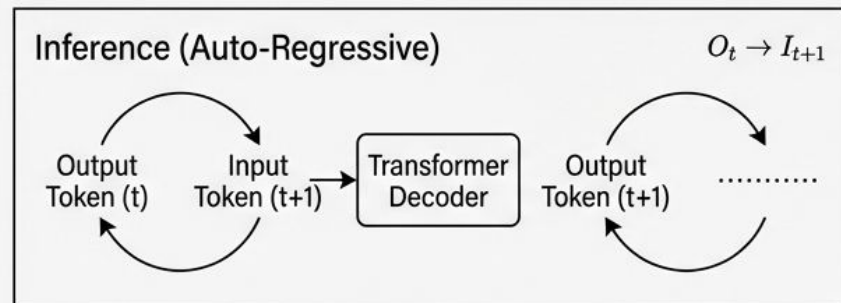
- Masked Self-Attention: Understands the "output so far" without peaking at the future.
- Add & Norm: Stabilizes the data.
- Cross-Attention: Connects the output-in-progress to the original input context.
- Add & Norm: Stabilizes again.
- Feed-Forward Network: Processes each word's representation independently.
- Add & Norm: Final stabilization before moving to the next layer or the final output.



## **Training Decoder Architecture: Parallel**

# Training vs. Inference: Architectural Behavior

## Operational Modalities



## Architectural Insight

The Transformer Decoder exhibits distinctly different operational modalities depending on the phase of the model's lifecycle. We are examining the Training Phase.

**The Training Advantage:** By utilizing the known ground-truth target sequence during training, the Decoder can process all tokens in parallel. This eliminates the sequential bottleneck, leveraging matrix multiplication for massive computational efficiency.

### Our Operating Example

**Encoder Input:** We are friends (English)

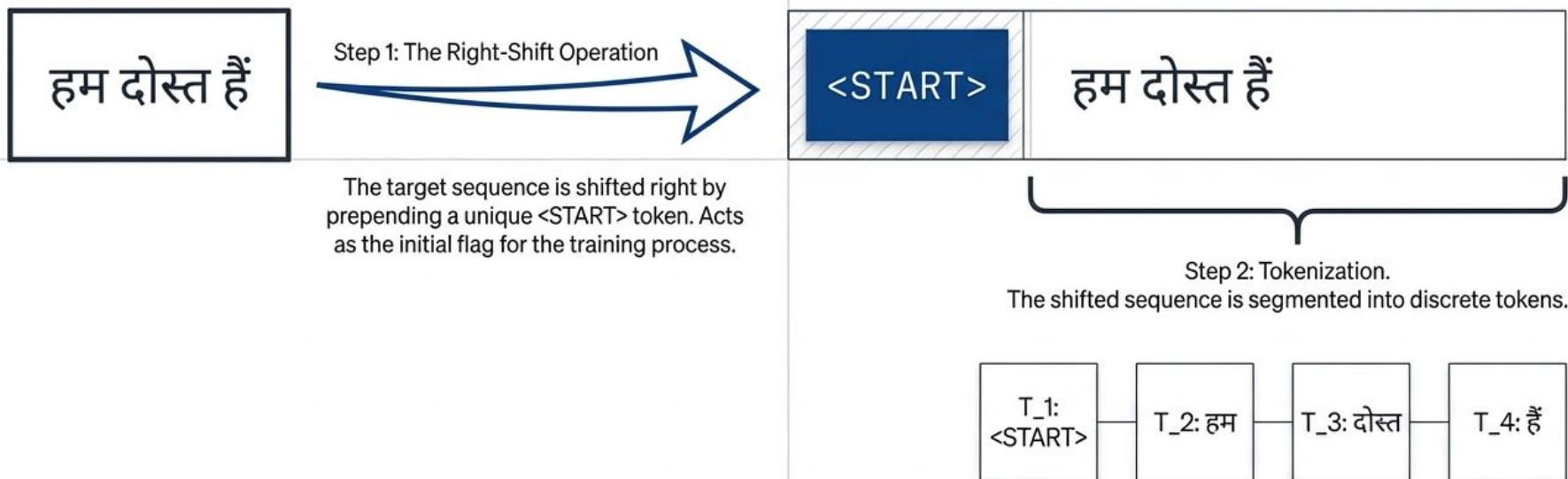
**Decoder Target:** हम दोस्त हैं (Hindi)

*Prerequisite: The Encoder has already generated 512-dimensional contextual embeddings for the English input.*

# Phase 1: Target Sequence Preparation

## Input Processing: Right-Shift & Tokenization

Before mathematical operations occur, the raw target text must be structured for the neural network.



Sequence Length (L): 4 tokens

# Translating Tokens to Tensor Space

Current Tensor Shape: [4 x 512]

Neural networks require continuous vector representations. Discrete tokens are mapped to a high-dimensional space.

## 1. Token Embedding Layer

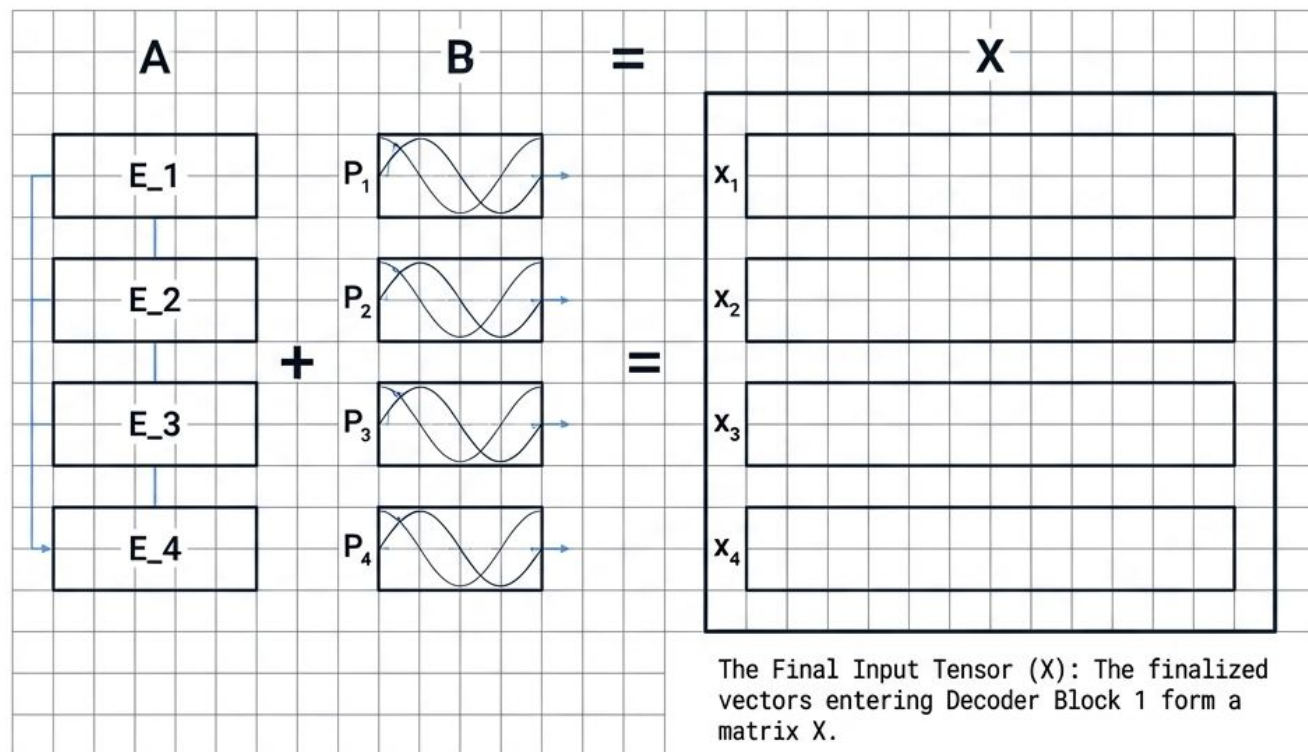
Each token is mapped to a continuous vector via a learned embedding matrix.

$E_1, E_2, E_3, E_4$  in  $\mathbb{R}^{512}$ .

Limitation: These embeddings lack sequence order awareness.

## 2. Positional Encoding

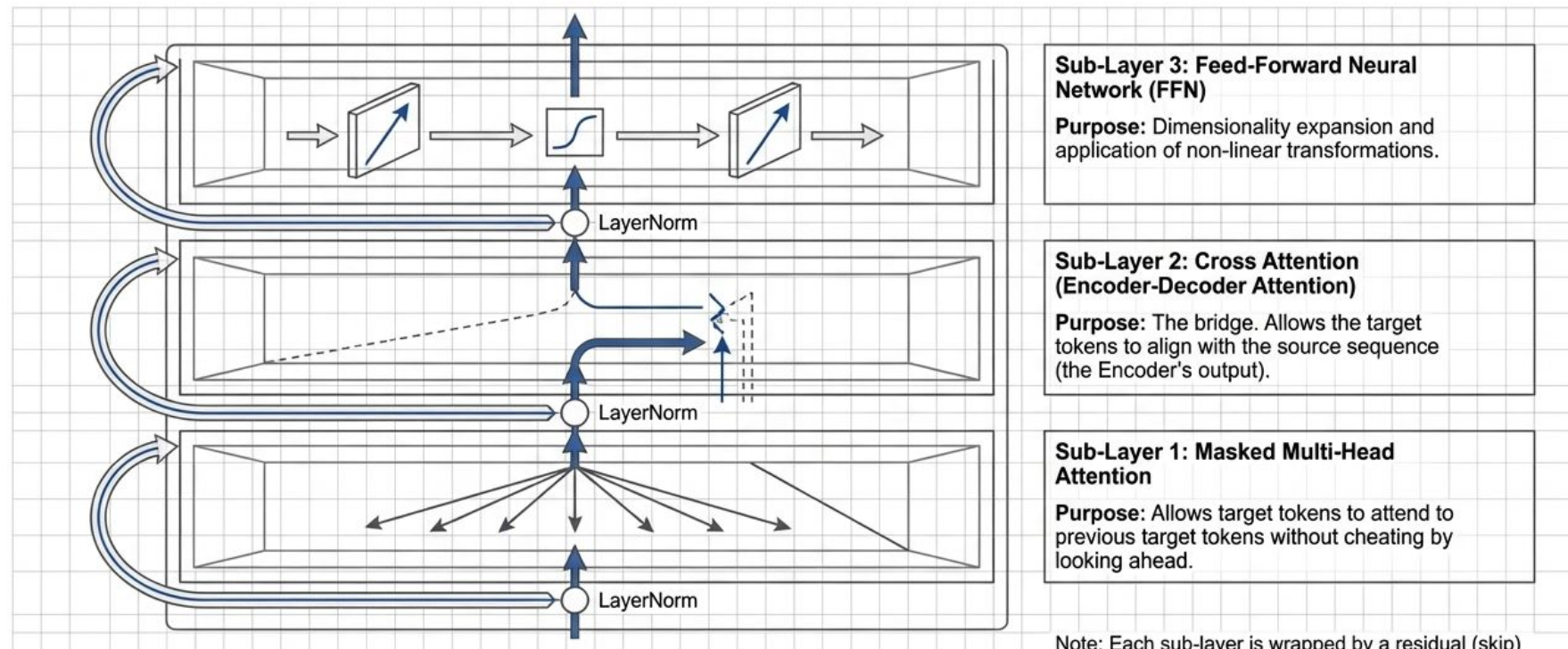
To inject sequence order, deterministic positional vectors (using sine/cosine functions) are generated for each discrete position ( $P_1, P_2, P_3, P_4$ ).





# The Tri-Partite Sub-Layer Architecture

A single Decoder block is significantly more complex than an Encoder block. It contains three primary sub-layers, each responsible for a distinct cognitive function.





# Masked Multi-Head Attention: Preventing Data Leakage

Input:  $[4 \times 512]$   $\rightarrow$   
Output:  $[4 \times 512]$

During parallel training, the entire target sequence is processed at once.

To simulate real-world sequential generation, the model must be artificially blinded to future tokens.

## The Causal Mask Mechanism:

- $x_1$  (<START>): Can only attend to  $x_1$ .
- $x_2$  (हम): Attends to  $x_1$  and  $x_2$ .
- $x_3$  (दोस्त): Attends to  $x_1$ ,  $x_2$ , and  $x_3$ . (Masked from  $x_4$ ).
- $x_4$  (हैं): Attends to the entire sequence.

|       | $x_1$ | $x_2$    | $x_3$    | $x_4$    |
|-------|-------|----------|----------|----------|
| $x_1$ |       | $\infty$ | $\infty$ | $\infty$ |
| $x_2$ |       |          | $\infty$ | $\infty$ |
| $x_3$ |       |          |          | $\infty$ |
| $x_4$ |       |          |          |          |

Output Generation: The sub-layer processes input matrix  $X$  and outputs intermediate contextualized vectors  $z_1, z_2, z_3, z_4$ .





# The Stabilization Protocol: Add & Norm

Immediately following every sub-layer operation, the architecture enforces a stabilization protocol.

## 1. The Add (Residual Connection)

The original input to the sub-layer is bypassed and added element-wise to the sub-layer's output.

$$\text{Equation: } Z'_i = x_i + z_i$$

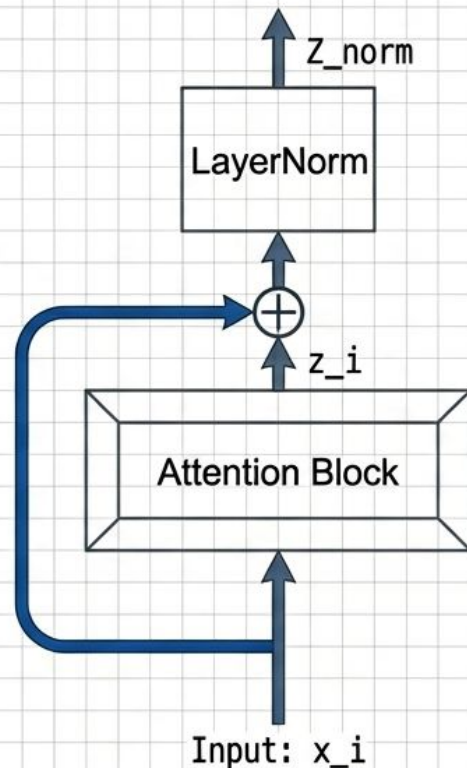
Function: Mitigates the vanishing gradient problem in deep networks.

## 2. The Norm (Layer Normalization)

Computes the mean ( $\mu$ ) and standard deviation ( $\sigma$ ) for each vector independently.

Function: Constrains vector values to a stable range, preventing extreme numerical fluctuations during dense matrix multiplications.

Result: Normalized vectors  $Z_{\text{norm}}$  advance to the next stage.





# Cross-Attention: The Linguistic Bridge

The Cross-Attention (Encoder-Decoder Attention) sub-layer is where translation occurs. It merges the contextual understanding of the English source with the unfolding Hindi target.

**The Origins of Q, K, and V:** Unlike Self-Attention, Cross-Attention bifurcates its data sources.

Queries (**Q**)

**Q**



Sourced from  
Decoder  
(Z\_norm)

Represents: What Hindi concept  
am I currently looking for?

Keys (**K**)

Sourced from  
Encoder  
Output



**K**

Represents: What English  
concepts do I contain?

Values (**V**)

Sourced from  
Encoder  
Output



**V**

Represents: The actual contextual  
meaning of the English words.

# Information Fusion in Cross-Attention

Input:  $[4 \times 512]$  & Encoder Output  
-> Output:  $[4 \times 512]$

By computing the dot product between the Decoder's Queries and the Encoder's Keys, the network calculates cross-lingual alignment scores.

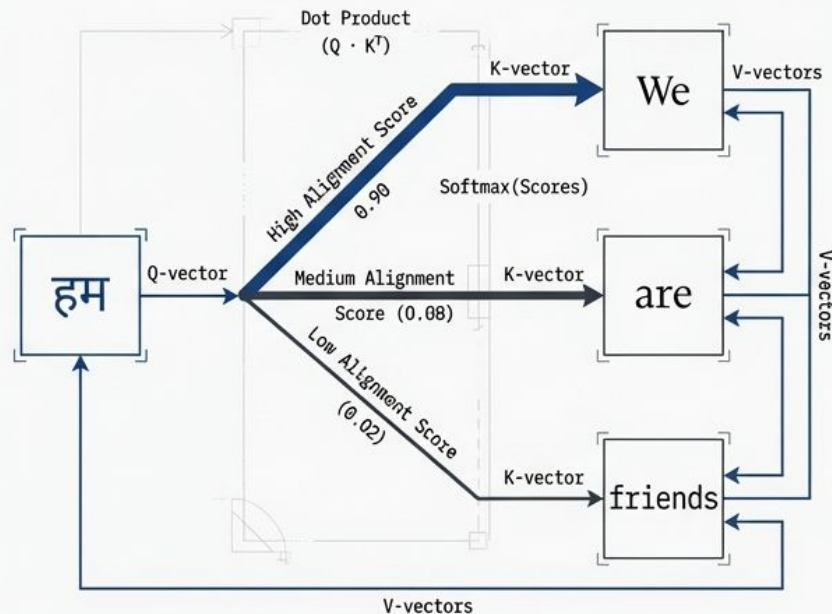
## The Mechanism:

- 1. Alignment:** A Hindi token's Query vector measures similarity against every English token's Key vector.
- 2. Weighting:** Softmax is applied to these scores to create normalized attention weights.
- 3. Extraction:** The English Value (V) vectors are multiplied by these weights and summed together.

## The Resulting Tensors:

We obtain a new set of vectors ( $zc_1, zc_2, zc_3, zc_4$ ) representing the Hindi tokens perfectly contextualized with relevant English translations. Followed by a second Add & Norm to yield  $Zc_{norm}$ .

## Dot-Product Alignment Visualization

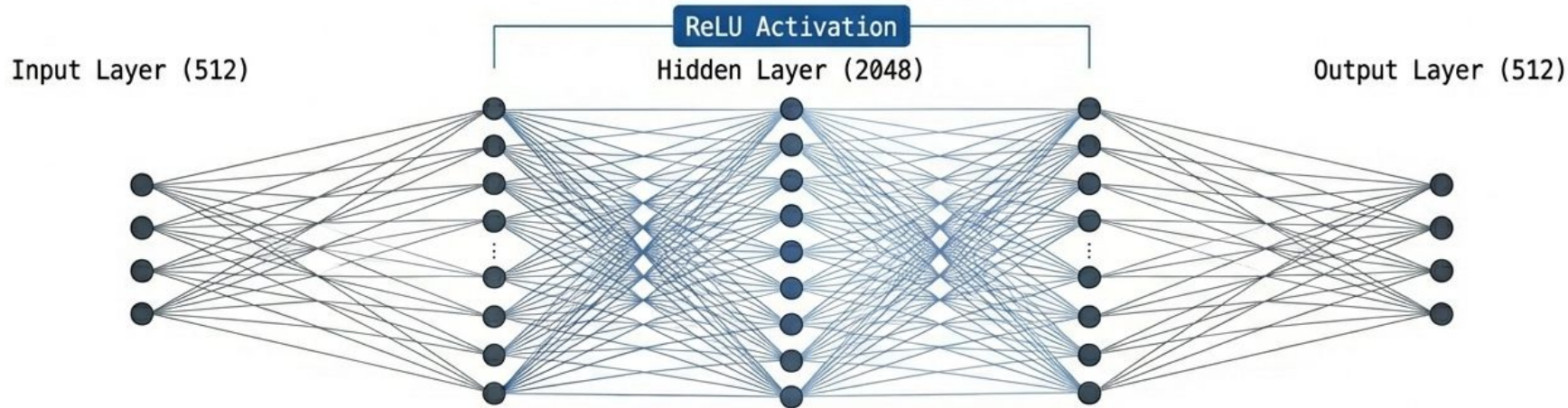






# Sub-Layer 3: The Feed-Forward Network (FFN)

The final sub-layer is a position-wise Feed-Forward Neural Network. The identical network is applied independently to each of the 4 token vectors.



## FFN Architecture:

Layer 1 (Expansion): 512 neurons  $\rightarrow$  2048 neurons.  
Activation: ReLU (Rectified Linear Unit).

Layer 2 (Compression): 2048 neurons  $\rightarrow$  512 neurons.  
Activation: Linear.

## The Purpose of the FFN:

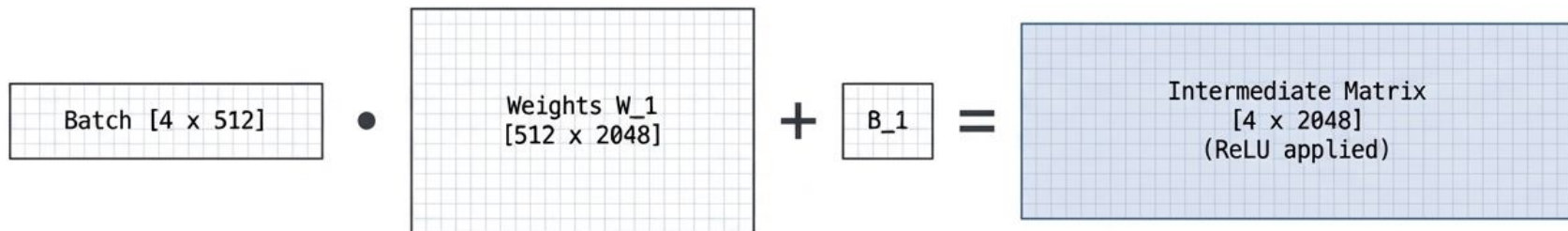
While Attention layers route information and find relationships between tokens, the FFN transforms the data. The massive expansion to 2048 dimensions, combined with ReLU, allows the network to learn complex, non-linear representations that Attention alone cannot capture.

# Batch Processing the FFN

Input:  $[4 \times 512] \rightarrow [4 \times 2048] \rightarrow \text{Final: } [4 \times 512]$

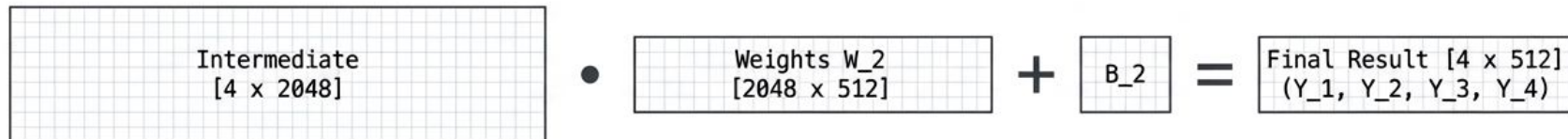
During execution, the 4 individual token vectors are processed simultaneously as a single batch matrix.

## Transformation 1 (Expansion)



Operation:  $(\text{Batch} \cdot W_1) + B_1$

## Transformation 2 (Compression)



Operation:  $(\text{Intermediate} \cdot W_2) + B_2$

*Output passes through the final Add & Norm to produce  $Y_{\text{norm}}$ .*

# Ascending the Decoder Stack

The vectors emerging from the first Decoder block have been thoroughly processed, but the architectural journey requires deep refinement.

## Sequential Propagation:

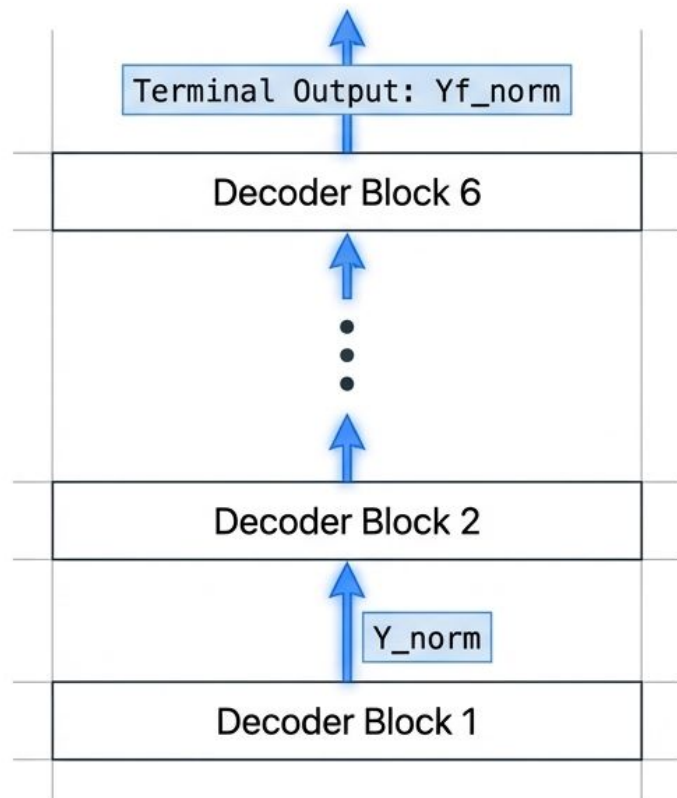
Block 2:  $Y_{\text{norm}}$  acts as the direct input (X) for Decoder Block 2. The exact same three sub-layers are applied, utilizing Block 2's unique, learned parameters.

Iterative Deepening: This cycle repeats strictly through Blocks 3, 4, and 5.

The Terminal Output: Upon exiting Decoder Block 6, we obtain the finalized representation matrices:  $Yf_1$ ,  $Yf_2$ ,  $Yf_3$ ,  $Yf_4$ .

The Tensor Shape remains strictly  $[4 \times 512]$  throughout the entire vertical stack.

Constant Shape:  $[4 \times 512]$





# Phase 3: The Output Head

Input:  $[4 \times 512]$  -> Output:  $[4 \times 10,000]$

## The Linear Output Layer: Mapping to Vocabulary

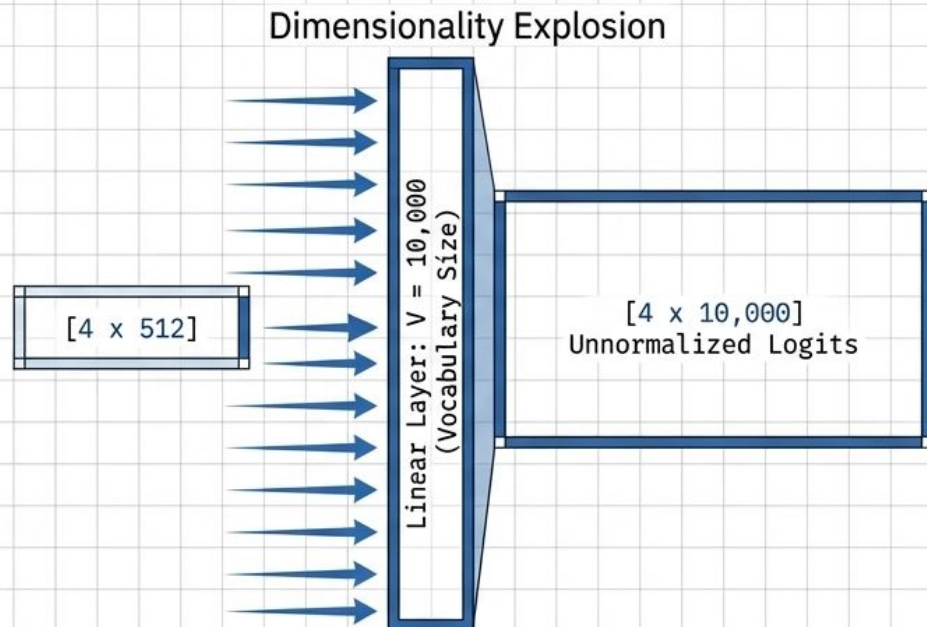
The finalized vectors from the 6th Decoder block hold deep linguistic representations, but remain abstract 512-dimensional arrays. They must be translated back into human vocabulary.

### The Linear Transformation:

- The output block utilizes a massive, single-layer fully connected network.
- **Input size: 512**
- **Output size ( $V$ ):** Equal to the total number of unique words in the target language training dataset (e.g.,  $V = 10,000$  unique Hindi words).
- **Parameters:** A weight matrix of  $[512 \times 10,000]$  plus  $10,000$  biases.

### The Output Matrix (Logits):

Multiplying our sequence batch  $[4 \times 512]$  by the Linear layer weights yields a final matrix of shape  $[4 \times 10,000]$ . Each token now possesses 10,000 raw, unnormalized scores.



# Softmax Normalization: Generating Predictions

The 10,000 raw numbers (logits) generated for each token are not probabilities; they can be negative or exponentially large.

## The Softmax Activation:

- Applied independently across the 10,000-dimensional vector for each token.
- **Function:** Exponentiates the logits and normalizes them so they sum precisely to 1.0.
- **Result:** A rigorous probability distribution over the entire vocabulary.

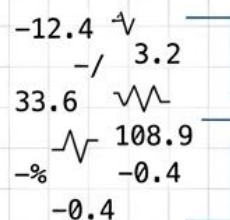
## Evaluating the Output (Example for Token 1):

- Out of 10,000 possible words, the model assigns probabilities:

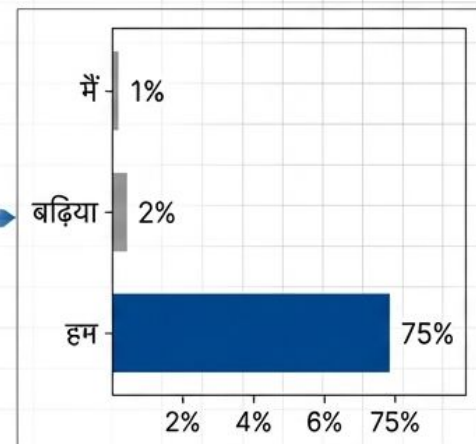
$$P(\text{मैं}) = 0.01$$

$$P(\text{बढ़िया}) = 0.02$$

$$P(\text{हम}) = 0.75 \text{ (Highest / Argmax)}$$

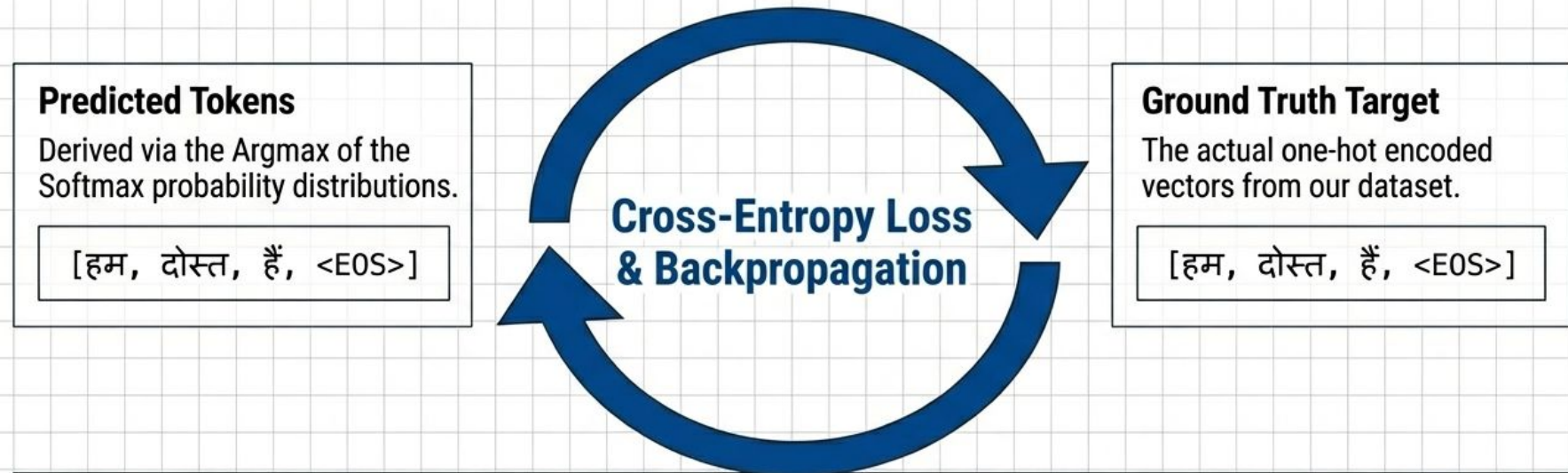


Raw Logits



# Closing the Loop: Loss Calculation

Because we are in the Training Phase, the architecture does not just print the output and stop. It must learn and optimize.

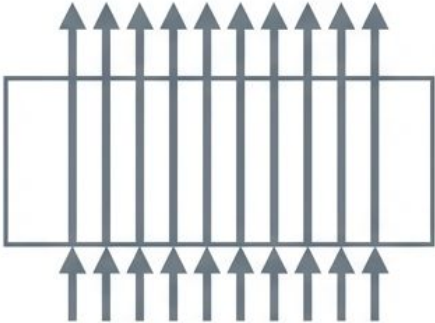
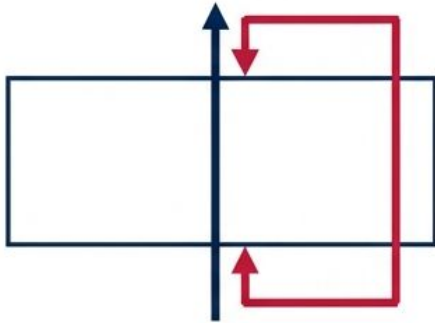


## The Gradient Update:

The probability distributions generated by the Softmax layer are rigorously compared against the ground truth using a Cross-Entropy Loss function. The resulting error signal is backpropagated through all 6 Decoder layers, the Encoder, and the Embeddings, optimizing the millions of parameters we just traversed.

# THE PARADIGM SHIFT: TRAINING VS. INFERENCE

## Behavioral Divergence in the Transformer Architecture

| TRAINING PHASE                                                                                                                                                                                                                                  | INFERENCE PHASE                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                |                                                                                                                                                                                                                                                                                    |
| <p>Encoder: Processes full input sequence simultaneously. <math>O(1)</math> time steps.</p> <p>Decoder: Non-Autoregressive. Processes the entire target sequence in parallel using teacher forcing. Look-ahead masking is applied globally.</p> | <p>Encoder: <b>IDENTICAL</b>. Processes full input sequence simultaneously. Contextual representations are computed once and cached.</p> <p>Decoder: Auto-Regressive. Operates sequentially over discrete time steps <math>(t_1, t_2, \dots, t_n)</math>. Generates one token per forward pass. Output at <math>t_i</math> becomes input at <math>t_{i+1}</math>.</p> |

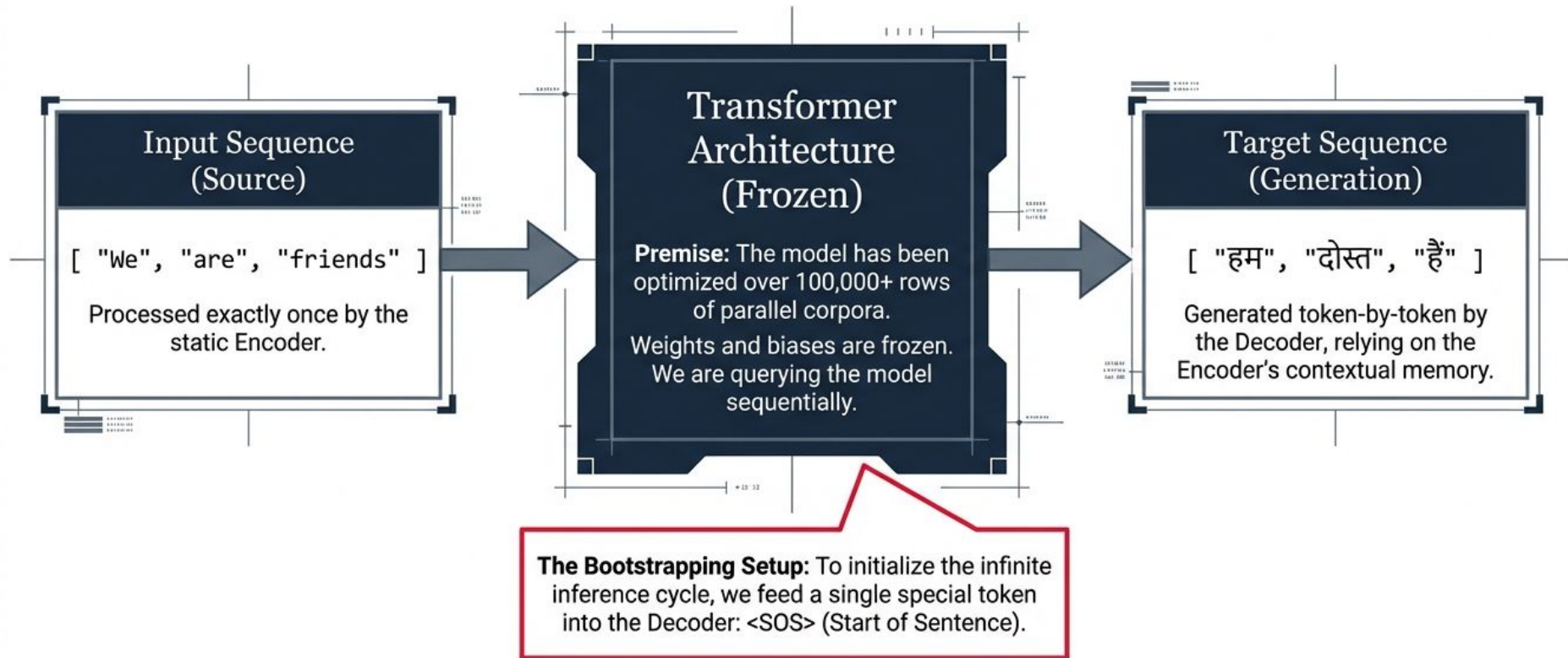
The fundamental engineering challenge of inference is managing the dynamically expanding input sequence within the Decoder block.

**Inference: Auto-Regressive**



# THE INFERENCE TASK FORMULATION

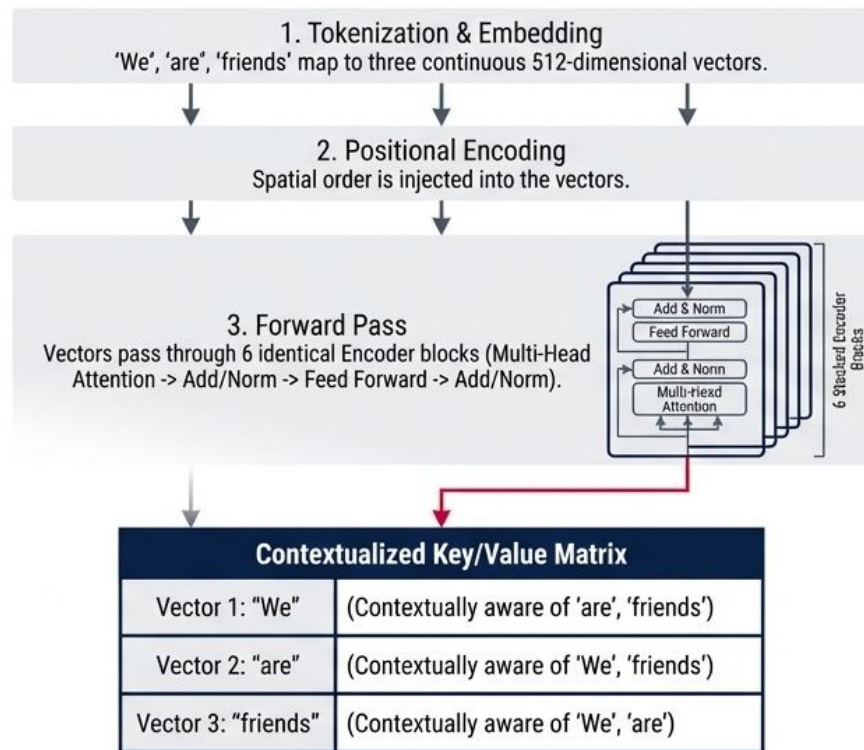
## Machine Translation: English to Hindi





# PHASE 1: ENCODER PROCESSING (STATIC OPERATION)

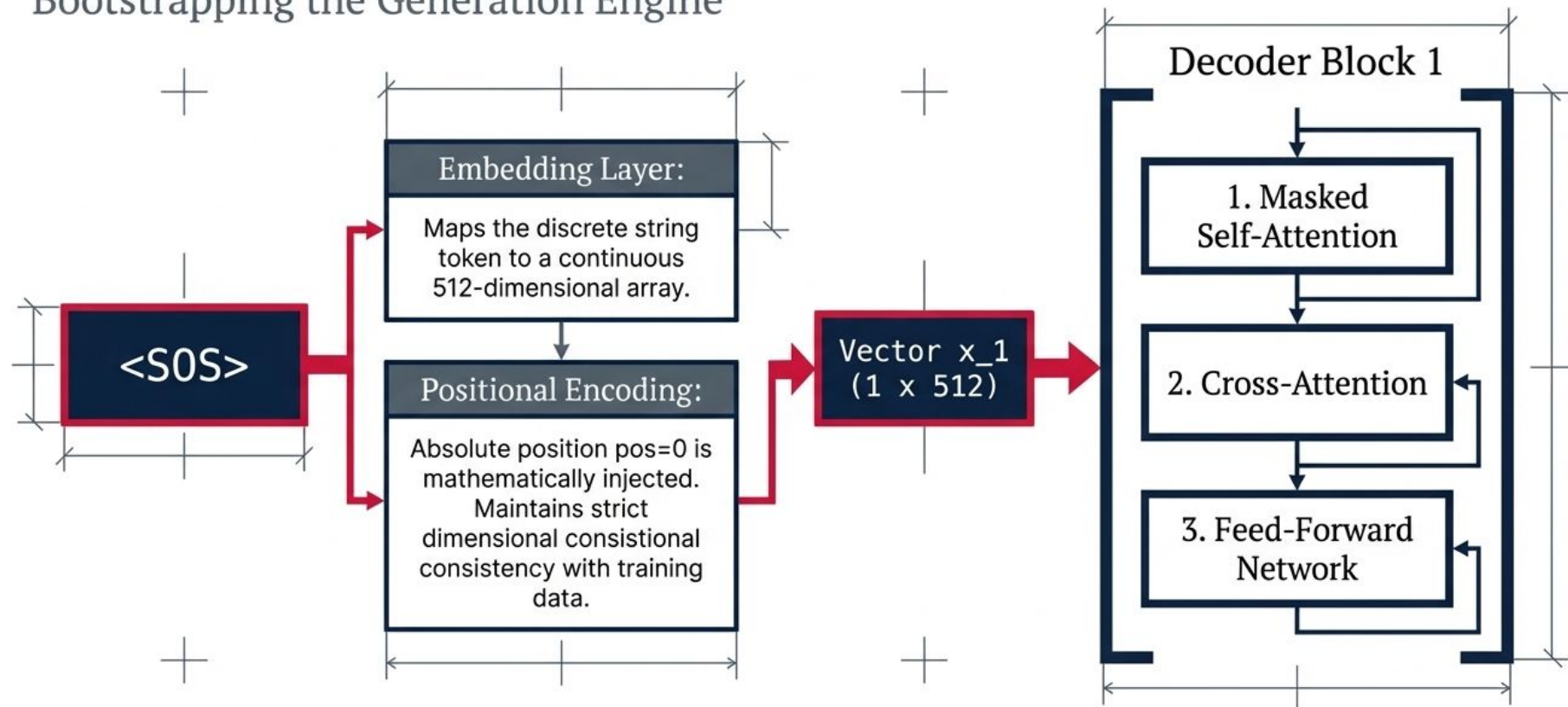
## Establishing the Contextual Foundation



*These vectors are now permanently cached. They will act as the static Keys (K) and Values (V) for every subsequent Decoder time step.*

# TIME STEP $t=1$ : DECODER INITIALIZATION

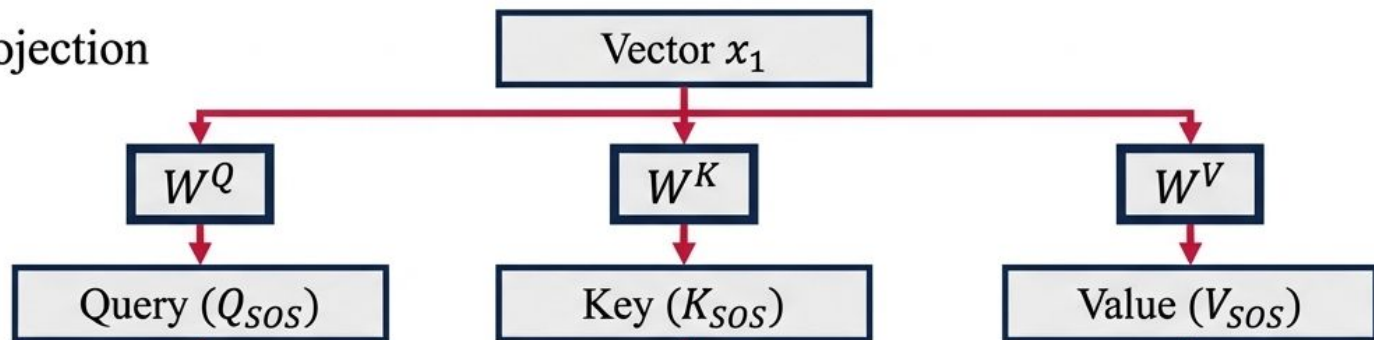
## Bootstrapping the Generation Engine



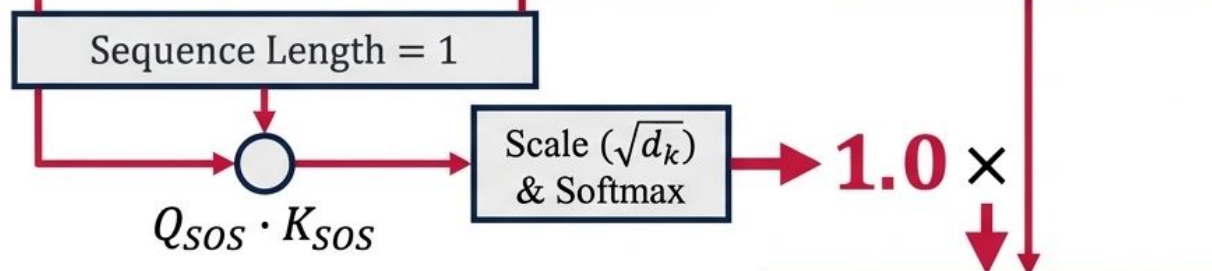
# DECODER $t=1$ : MASKED SELF-ATTENTION

Computing Internal Token Relationships

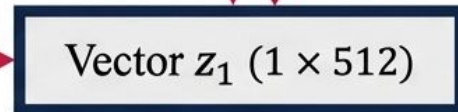
## 1. Dimensional Projection



## 2. Attention Scoring



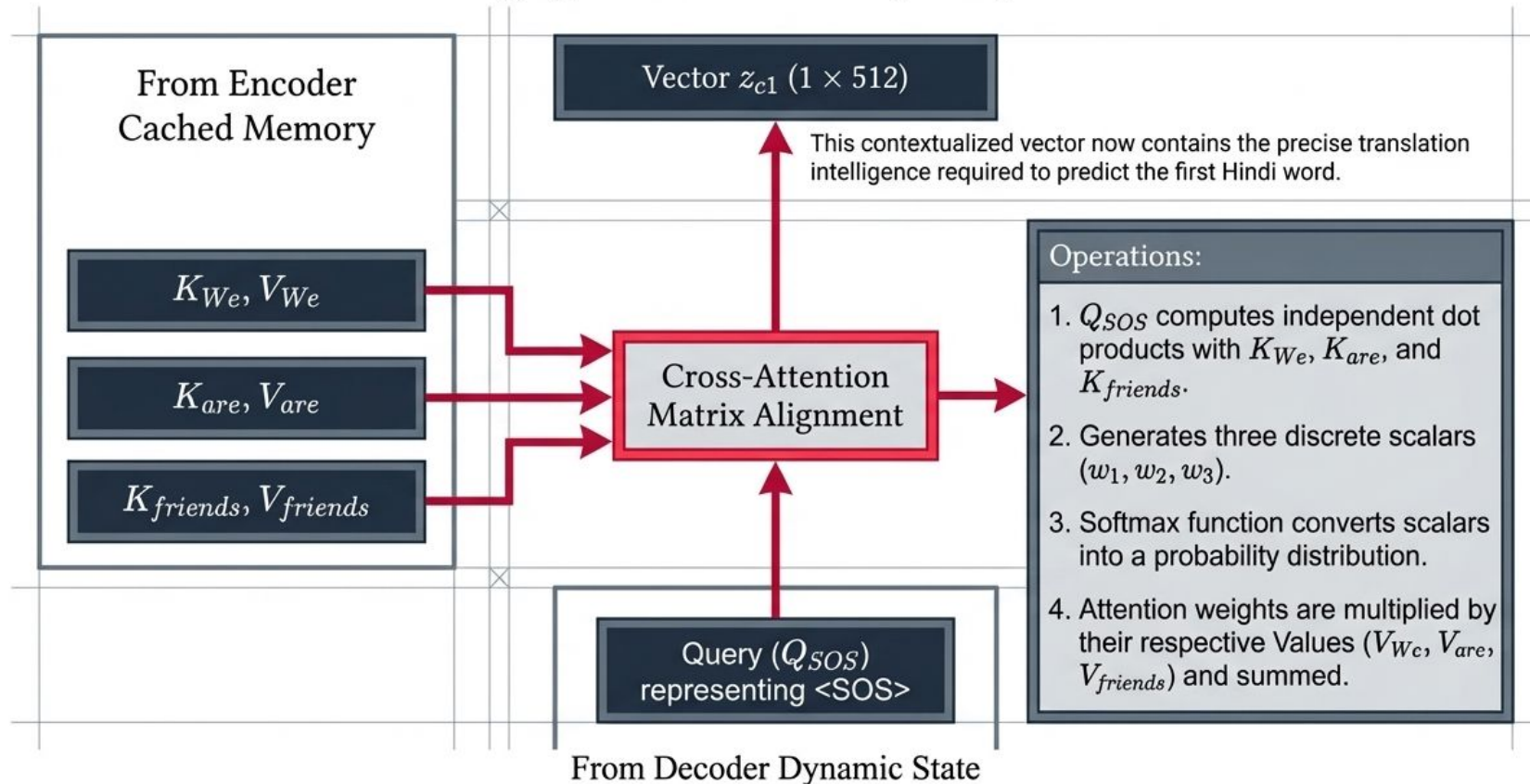
Output State



Post-computation,  $z_1$  is routed through a Residual Connection (Add & Norm) yielding the normalized output:  $z_1^{norm}$ . NotebookLM

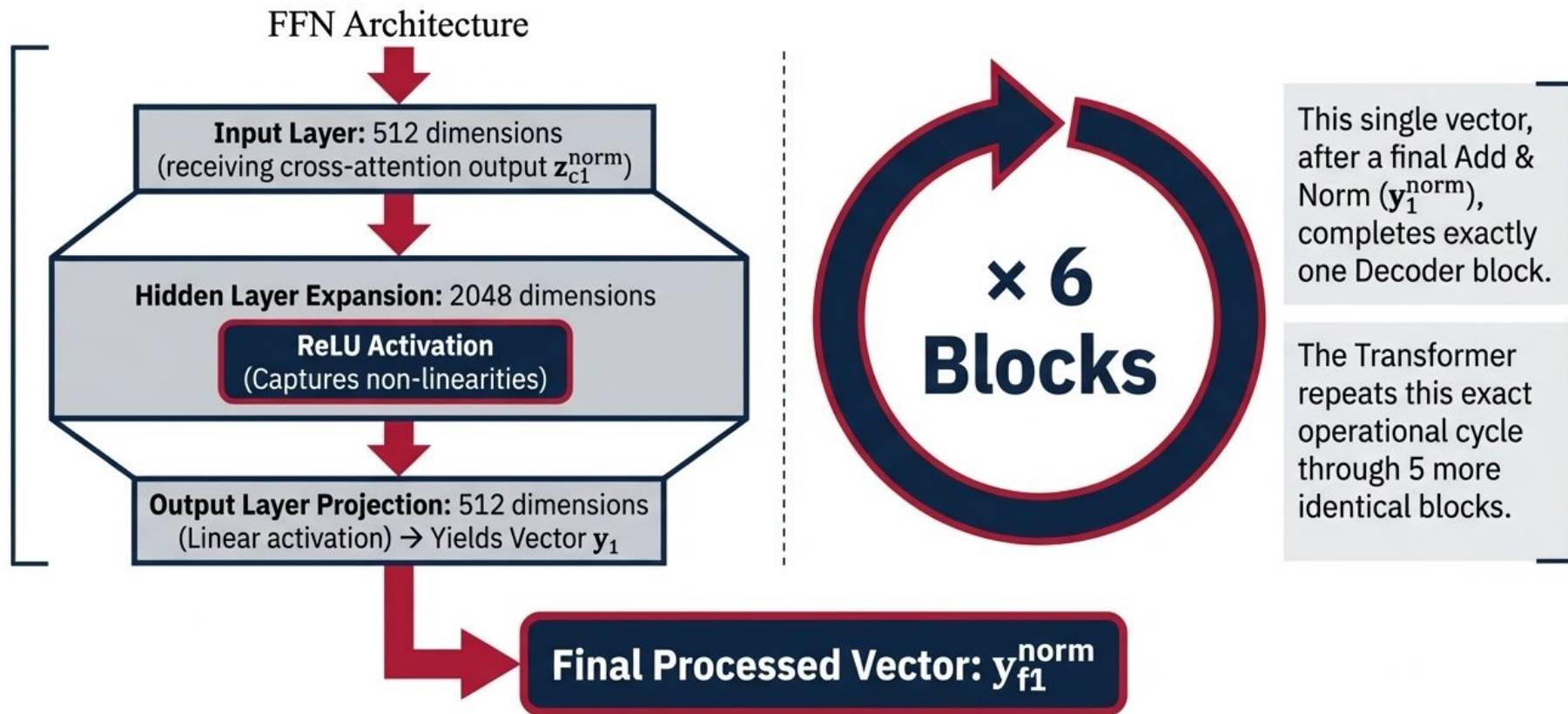
# DECODER $t=1$ : CROSS-ATTENTION COMPUTATION

Bridging the Source and Target Sequences



# DECODER $t=1$ : FEED-FORWARD & BLOCK ITERATION

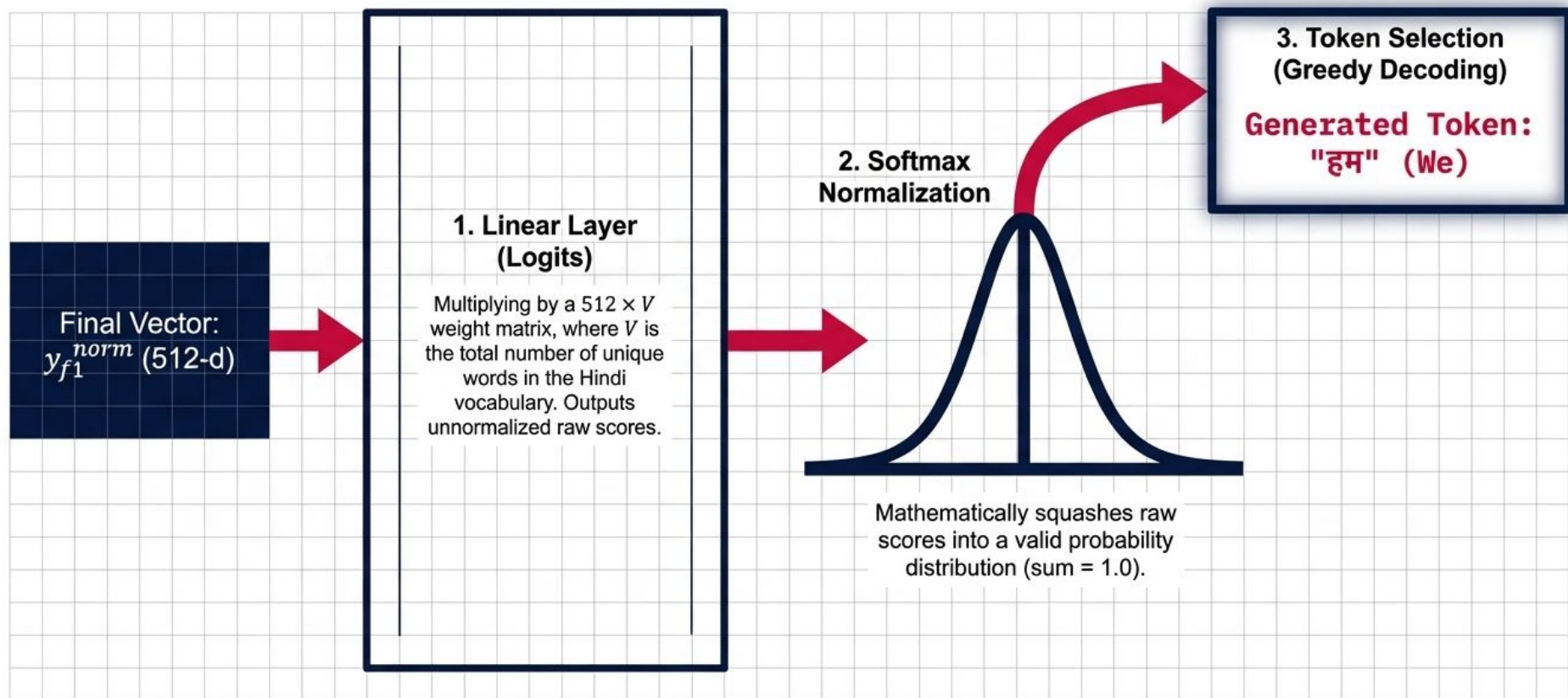
## Capturing Non-Linear Transformations





# DECODER $t=1$ : THE GENERATION HEAD

## Projecting to the Vocabulary Space



# TIME STEP $t=2$ : THE AUTO-REGRESSIVE FEEDBACK LOOP

Expanding the Input Sequence

Output token from  $t=1$

“हम”

## 1 The Feedback Mechanism

The defining characteristic of inference occurs here. The model does not move to the next word in isolation; it builds a cumulative historical sequence.

## 2 Vector Re-Initialization

Both tokens pass through Embedding and Positional Encoding from scratch, generating two discrete  $1 \times 512$  vectors:  $x_1$  and  $x_2$ .

New Input Sequence for  $t=2$ :  
[<SOS>, “हम”]

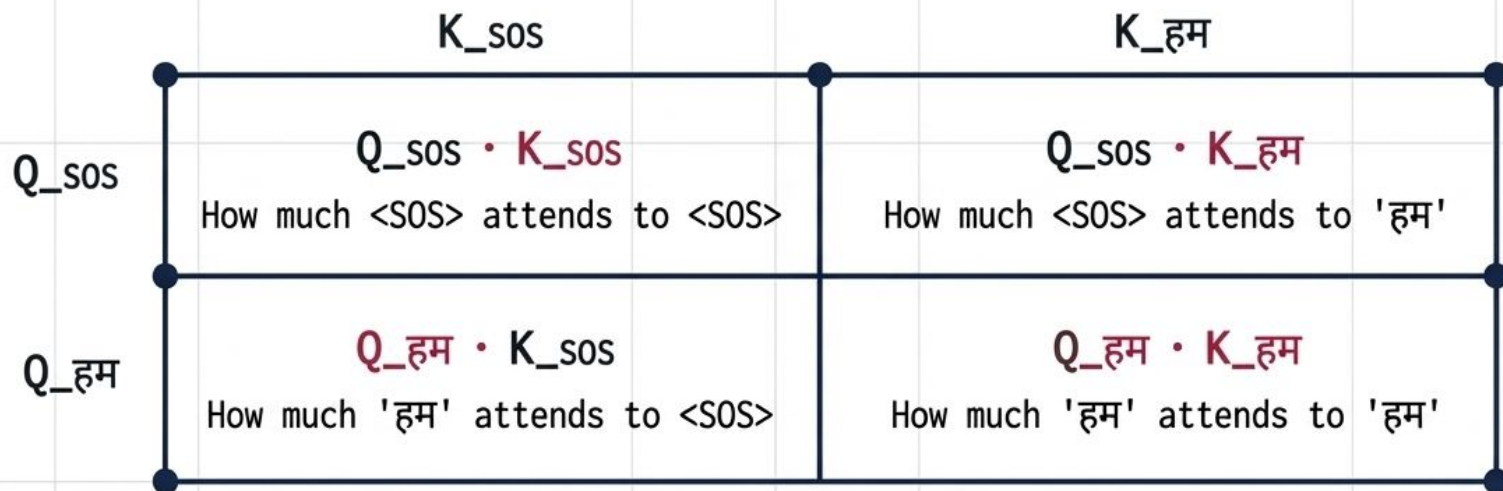
[<SOS>, “हम”]

**Crucial Engineering Constraint:** The Decoder must now process both vectors simultaneously, recalculating all matrix relationships entirely from scratch.

# DECODER $t=2$ : EXPANDED SELF-ATTENTION

## Computing Multi-Token Dynamics

At  $t=2$ , the block receives two distinct vectors ( $x_1$ ,  $x_2$ ), generating two sets of Queries, Keys, and Values. It calculates a  $2 \times 2$  attention matrix via four distinct dot products:



Before applying Softmax to normalize these scores, the model enforces a strict, counter-intuitive architectural constraint: **Inference Masking**.

# THE INFERENCE MASKING IMPERATIVE

Why do we mask during Inference?

Modified - matrix based on previous slide

|                  | $K_{\text{SOS}}$                                                           | $K_{\text{हम}}$                                                                                       |
|------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| $Q_{\text{SOS}}$ | $Q_{\text{SOS}} \cdot K_{\text{SOS}}$<br>(How much <SOS> attends to <SOS>) | <del><math>Q_{\text{SOS}} \cdot K_{\text{हम}}</math><br/>(How much &lt;SOS&gt; attends to 'हम')</del> |
| $Q_{\text{हम}}$  | $Q_{\text{हम}} \cdot K_{\text{SOS}}$<br>(How much 'हम' attends to <SOS>)   | $Q_{\text{हम}} \cdot K_{\text{हम}}$<br>(How much 'हम' attends to 'हम')                                |

## The Operation:

The upper triangular values of the dynamic attention matrix (e.g., <SOS> looking forward at 'हम') are forcefully set to  $-\infty$ , driving their Softmax probabilities to **precisely 0**. <SOS> is mathematically blinded to the future token.

## The Mathematical Necessity (The "Why"):

**Misconception:** Look-Ahead Masking is only used during training to prevent data leakage.

**Reality:** Masking is **strictly enforced** during inference. The model optimized its weights under the strict condition of **masked matrices**.

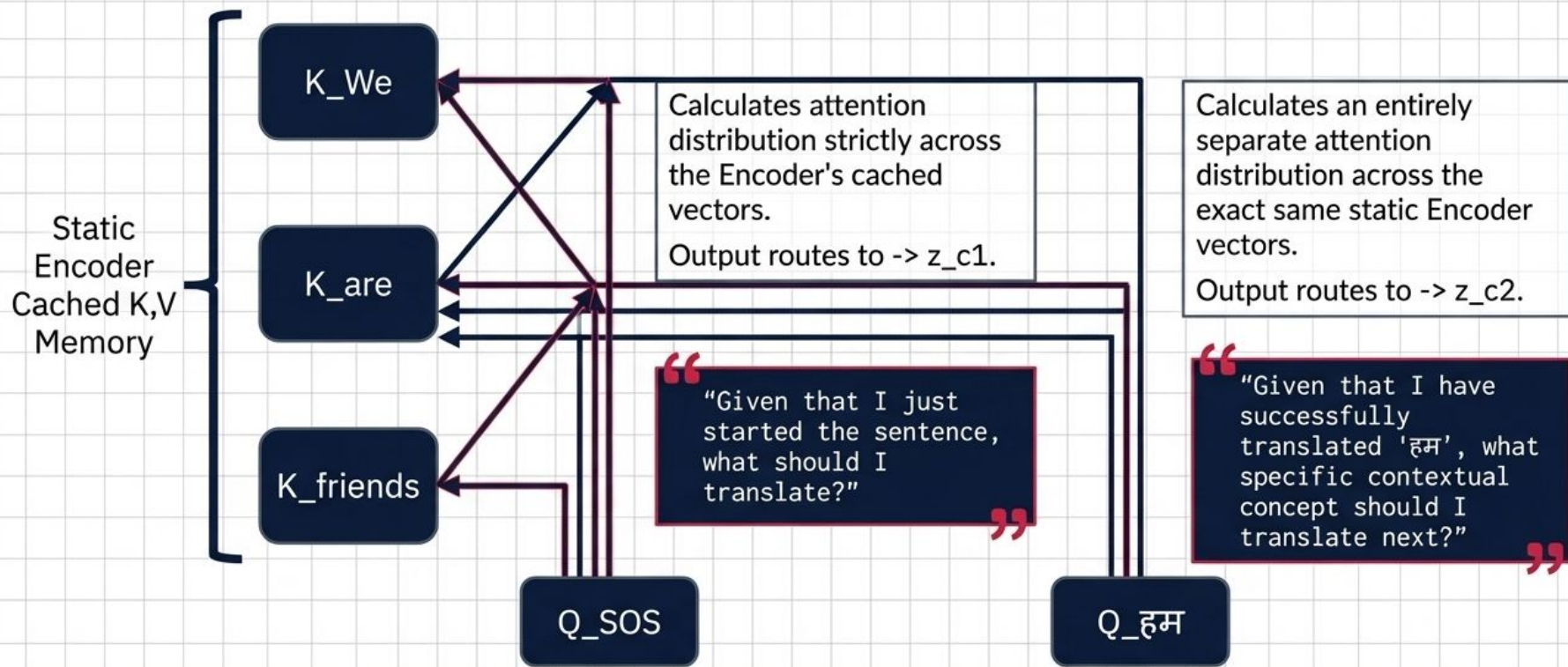
If we remove the mask simply because we 'already know' the generated token, the mathematical distribution of the resulting vectors **shifts dramatically**.

Applying the mask ensures **strict distributional consistency** between the training environment and the real-world prediction environment.



# DECODER $t=2$ : PARALLEL CROSS-ATTENTION

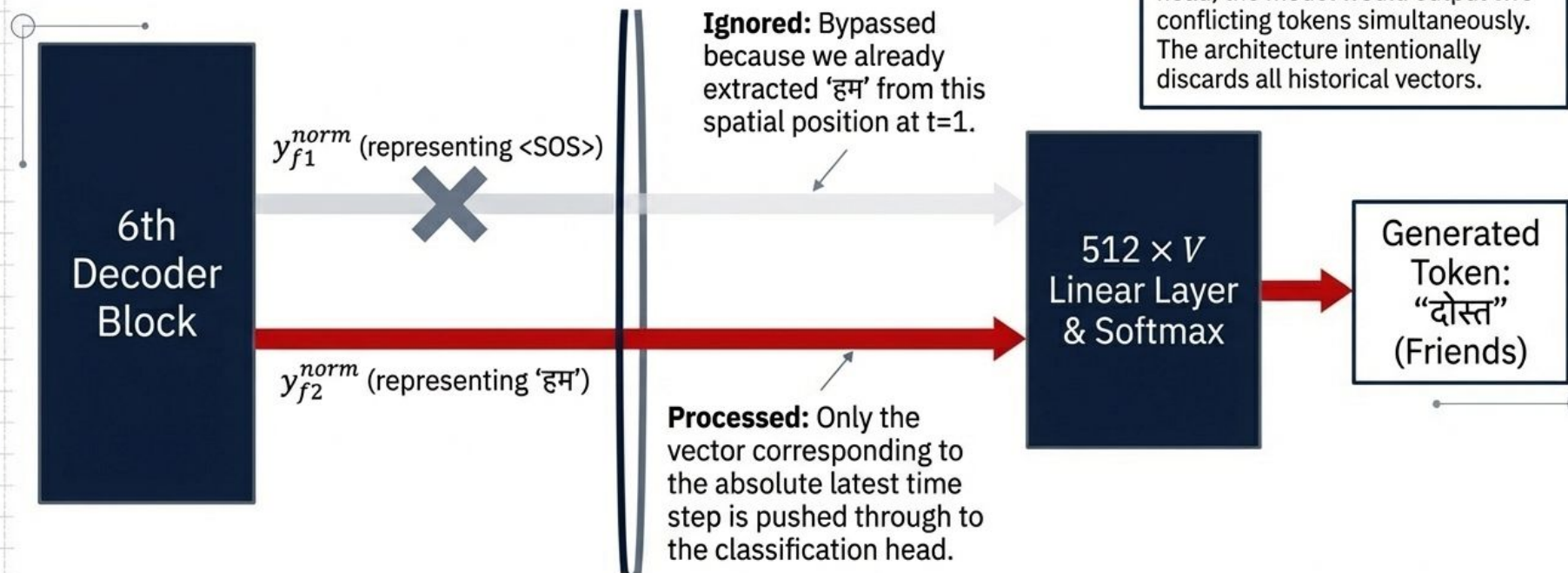
Mapping the Expanding Sequence to Static Context





# THE OUTPUT SELECTION RULE

## Extracting the Current Time Step



# TIME STEP $t=N$ : SEQUENCE EXPANSION & TERMINATION

## *The Computational Cost of Inference*

t=1 Input: [ <SOS> ] → Generates 'हम'

t=2 Input: [ <SOS>, 'हम' ] → Generates 'दोस्त'

t=3 Input: [ <SOS>, 'हम', 'दोस्त' ] → Generates 'हैं'

t=4 Input: [ <SOS>, 'हम', 'दोस्त', 'हैं' ] → Evaluates termination...

**The Expanding Horizon:**  
 $O(N^2)$  Complexity.

The mathematical process repeats exactly, but computational complexity scales quadratically with each time step due to expanding Attention matrices.

<EOS>

**The Termination Condition:** The auto-regressive loop is structurally infinite. The Transformer engine only halts when the final Softmax distribution assigns the absolute highest probability to the strictly defined <EOS> (End of Sentence) token.

**Final Extracted Output:** 'हम दोस्त हैं'

# SYNTHESIS: DECODER MECHANICS MATRIX

## Architectural State Comparison

| System Dimension         | Training Phase                                                                                | Inference (Prediction) Phase                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Execution Mode           | Parallel. Entire sequence processed in a single, simultaneous forward pass.                   | <b>Auto-Regressive</b> . Sequential loop; N discrete forward passes to generate N tokens.                                  |
| Input State              | Complete ground truth target sequence (Teacher Forcing).                                      | <b>Dynamically expanding array</b> of historical predictions.                                                              |
| Self-Attention Masking   | Look-Ahead Masking. Applied to prevent data leakage (cheating on future words).               | Distributional Masking. Applied strictly to maintain mathematical consistency with trained weights.                        |
| Generation Head          | All sequence vectors passed to Linear layer simultaneously to calculate global gradient loss. | <b>Selective Extraction</b> . Only the absolute final sequential vector ( $y_{ft}^{norm}$ ) is passed to the Linear layer. |
| Computational Complexity | Highly optimized matrix parallelization. Fixed sequence lengths.                              | <b><math>O(N)</math> sequential bottleneck</b> . Quadratically expanding matrix dimensions.                                |

*Mastery of Transformer Inference lies entirely in managing this complex transition from parallelized matrix operations to dynamically expanding sequential loops.*

# Decoder Architecture: Autoregressive Generation (GPT)

**The Autoregressive Goal:** Predict the conditional distribution of each token given ONLY the prior tokens.

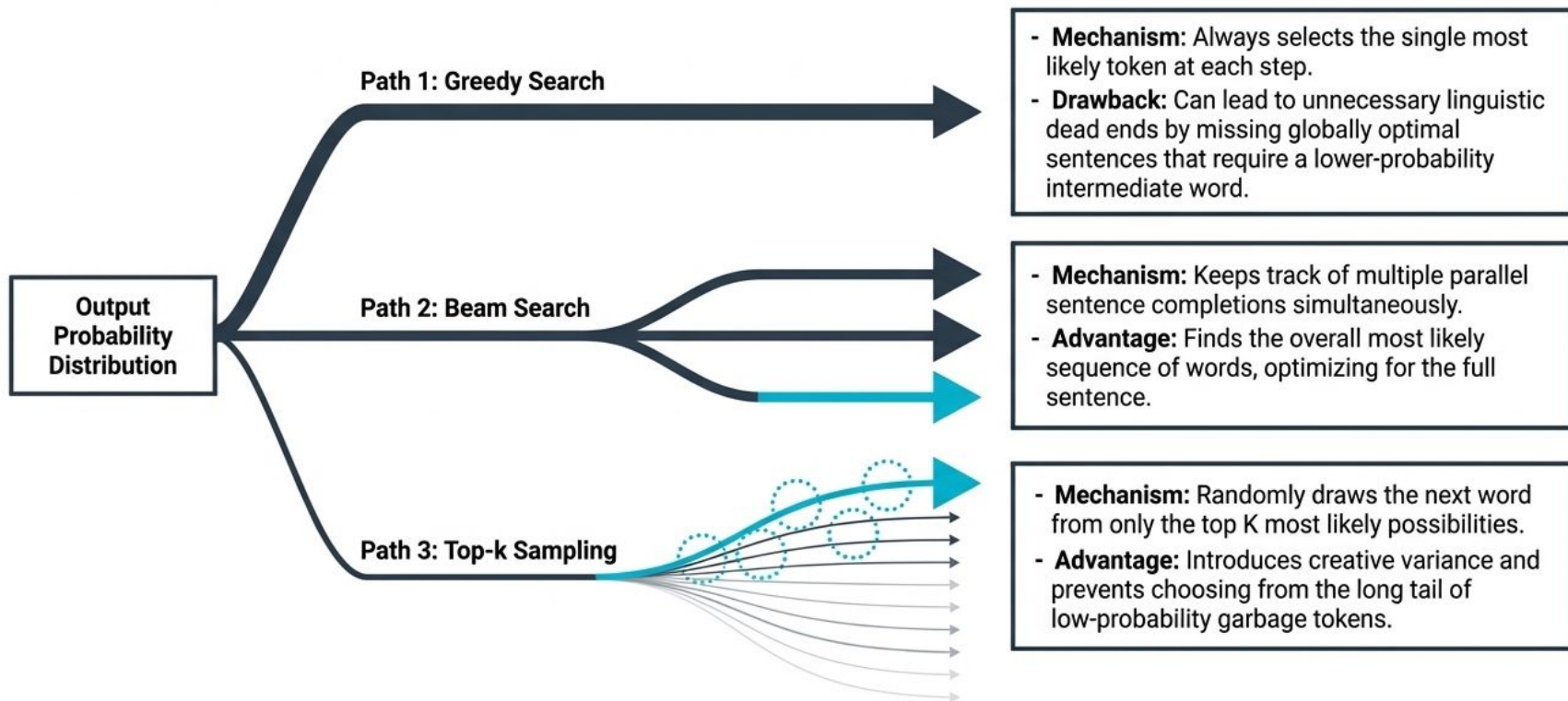
$$\Pr(t_1, t_2, \dots, t_N) = \Pr(t_1) \prod_{n=2}^N \Pr(t_n | t_1, \dots, t_{n-1})$$

$\Pr(\text{It})$   
 $\times \Pr(\text{takes} \mid \text{It})$   
 $\times \Pr(\text{great} \mid \text{It takes})$   
 $\times \Pr(\text{courage} \mid \text{It takes great})$   
 $\times \Pr(\text{to} \mid \text{It takes great courage})$   
 $\times \Pr(\text{let} \mid \text{It takes great courage to})$   
 $\times \Pr(\text{yourself} \mid \text{It takes great courage to let})$   
 $\times \Pr(\text{appear} \mid \text{It takes great courage to let yourself})$   
 $\times \Pr(\text{weak} \mid \text{It takes great courage to let yourself appear})$

*(This formulation proves that maximizing next-token prediction inherently maximizes the joint probability of the entire sequence).*



# Decoding Strategies for Text Generation





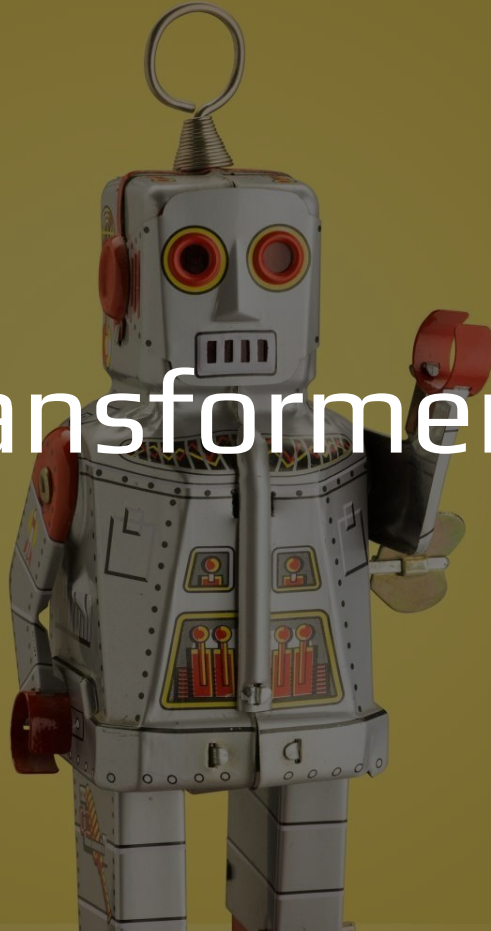
# Practical Applications

Cross attention is vital whenever a model must map relationships across two distinct modalities or sequences:

- **Machine Translation:** Aligning words between a source language and a target language.
- **Question Answering:** Determining which specific parts of a context paragraph are relevant to answering a prompt.
- **Image Captioning:** Treating the image and text as different sequences to generate descriptive text.
- **Text-to-Image Generation:** Translating textual descriptions into pixel relationships.

|       | We                                                                                  | are                                                                                 | friends                                                                             |
|-------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| हम    |  |  |  |
| दोस्त |  |  |  |
| गोप   |  |  |  |

# Transformers



# Problem Statement

- You have 1B parameter transformer.
- You need following models:
  - Legal assistant
  - Medical assistant
  - Code assistant
- Will you train 3 models from scratch?
- Why?

# Transfer learning

- Transferring a model trained on **source** data A to **target** data B.
- Rationale: Similar feature can be useful in different tasks,
- Or shared by different yet related data.
- There can be two possibilities:
  - Task transfer
  - Data transfer

# Task transfer

- The source and target data can be the same
- *Image classification* → *Image segmentation*
- *Machine translation* → *Sentiment analysis*
- *Time series prediction* → *Time series classification*



# Data transfer

- *Images of every objects → Medical images*
- *Chinese → English*
- *Physiological signals of one patient → Another patient*

# Transfer learning

Dog/Cat  
Classifier



cat



dog

Data **not directly related to** the task considered



elephant



tiger

Similar domain, different tasks



dog



cat

Different domains, same task

# Model fine-tuning

- Transfer a model trained on **large labeled source data** to very **little labeled target data**.
- Example: Speech recognition
  - Source: Audio data and transcriptions of many historical speaker
  - Target: Limited audio data and transcriptions of a new speaker.
- **Idea:**
  - Pre-train a model using labeled source data →
  - Fine-tune the model with labeled target data.